

Deep Learning for Computer Vision

Lecture 4: CNNs and Transfer Learning

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen



cppape

<https://user.informatik.uni-goettingen.de/~pape41/>

Agenda for today

- From last lecture:
 - EfficientNet + Neural Architecture Search
- Training large-scale CNNs
 - Learning rates, model ensembling, data augmentation
- Transfer learning
 - Visual saliency prediction (transfer learning example)

Organization

Submitting the exercises: please submit the exercise into the folder for your tutorial!

- Example: Your group is in “Tutorial 3” (Wednesday 16-18 with Oleg)
- Upload your solution to the folder called “Tutorial 3”



/ Submission of Homework 2

Den zeitgesteuerten Ordner sehen Teilnehmende in einem von lehrenden Personen bestimmten Zeitraum. Ob Teilnehmende die Ordnerinhalte herunterladen und/oder hochladen können, kann eingestellt werden.

Sichtbar von 11.11.2025 10:00 bis 25.11.2025 10:00

Dieser Ordner ist ein Hausaufgabenordner. Es können nur Dateien eingestellt werden.

Exercise 2

☐	Typ	Name ^	Größe	Autor/-in	Datum	Aktionen
☐		Tutorial 1		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 2		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 3		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 4		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 5		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 6		Archit, Anwai	11.11.2025 09:37	
☐		Tutorial 7		Archit, Anwai	11.11.2025 09:37	
HerunterladenVerschiebenKopierenLöschenNeuer OrdnerDokument hinzufügen						

Organization

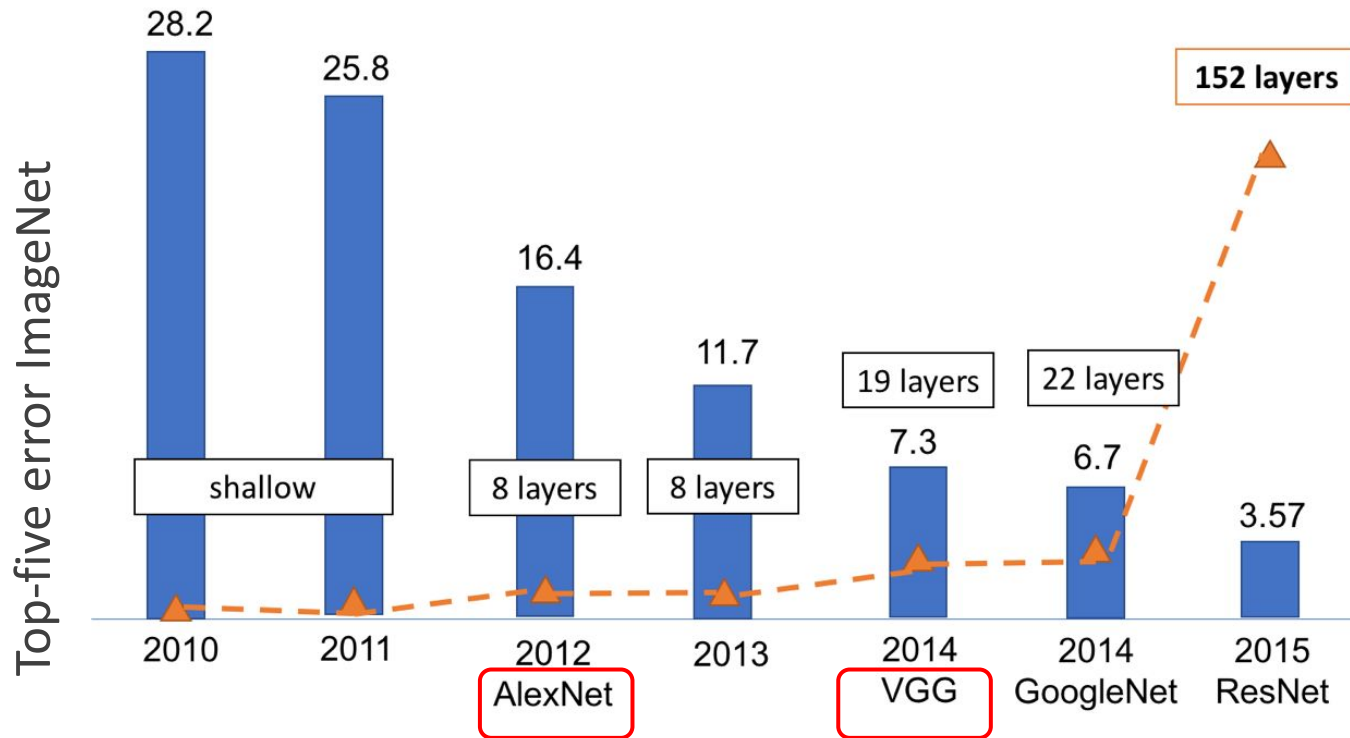
Using AI for working on the exercises:

- You are allowed to use ChatGPT or other LLM-based chat bots to support you with the exercises.
- BUT: you have to understand your code and solution!
- If tutors think that you don't understand your code (or if you are not good prepared) they can fail you — even if the solutions are all there.
- Tutors can also fail groups partially, i.e. only pass for the members that were well prepared and understood the solutions.

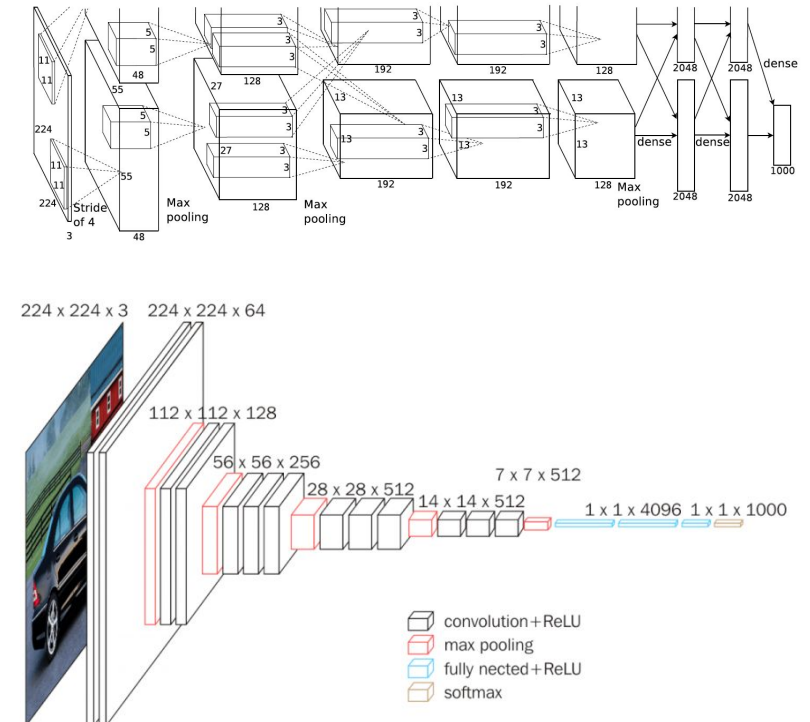
Recap CNN Architectures

+ EfficientNet and Neural Architecture Search

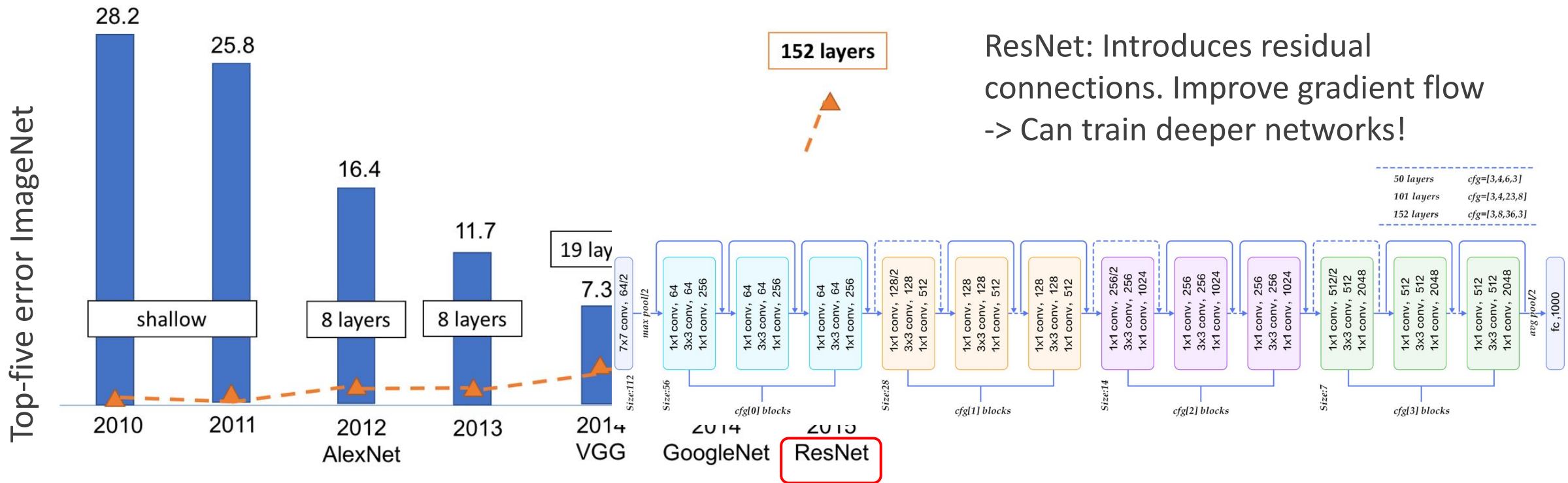
Recap CNN Architectures



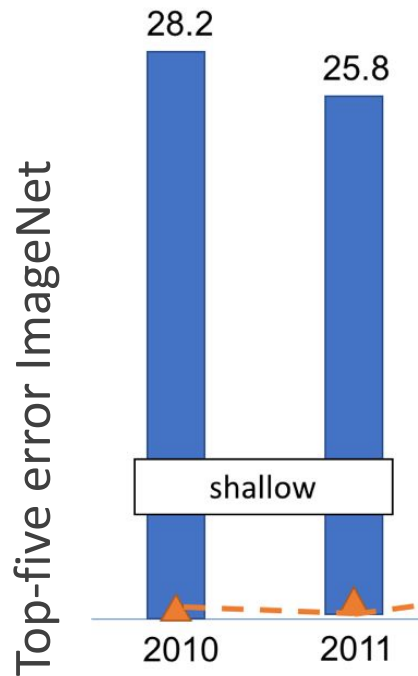
AlexNet + VGG: Standard CNNs



Recap CNN Architectures



Recap CNN Architectures

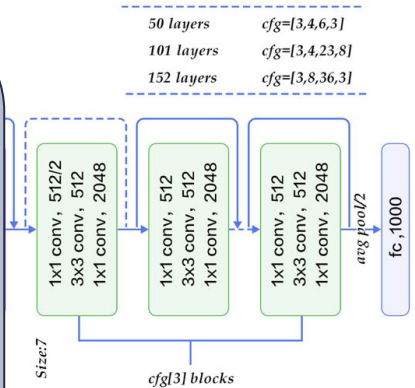


Not only accuracy is important! Efficiency also matters.

- **MobileNet: Make networks more efficient in inference to run on edge devices / smartphone.**
 - Uses depth-wise separable convolutions.

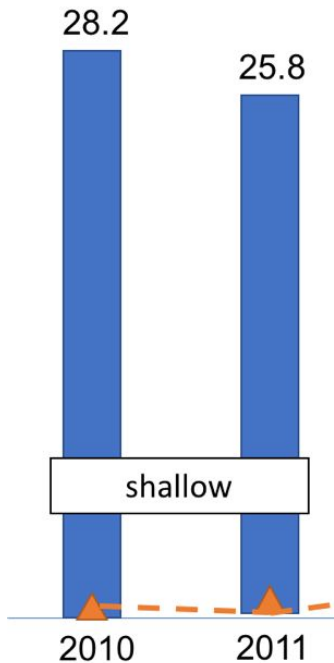
152 layers

ResNet: Introduces residual connections. Improve gradient flow
-> Can train deeper networks!



Recap CNN Architectures

Top-five error ImageNet

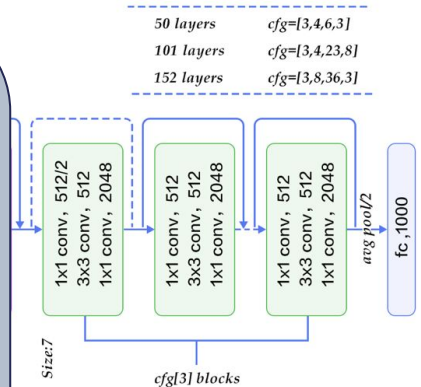


152 layers

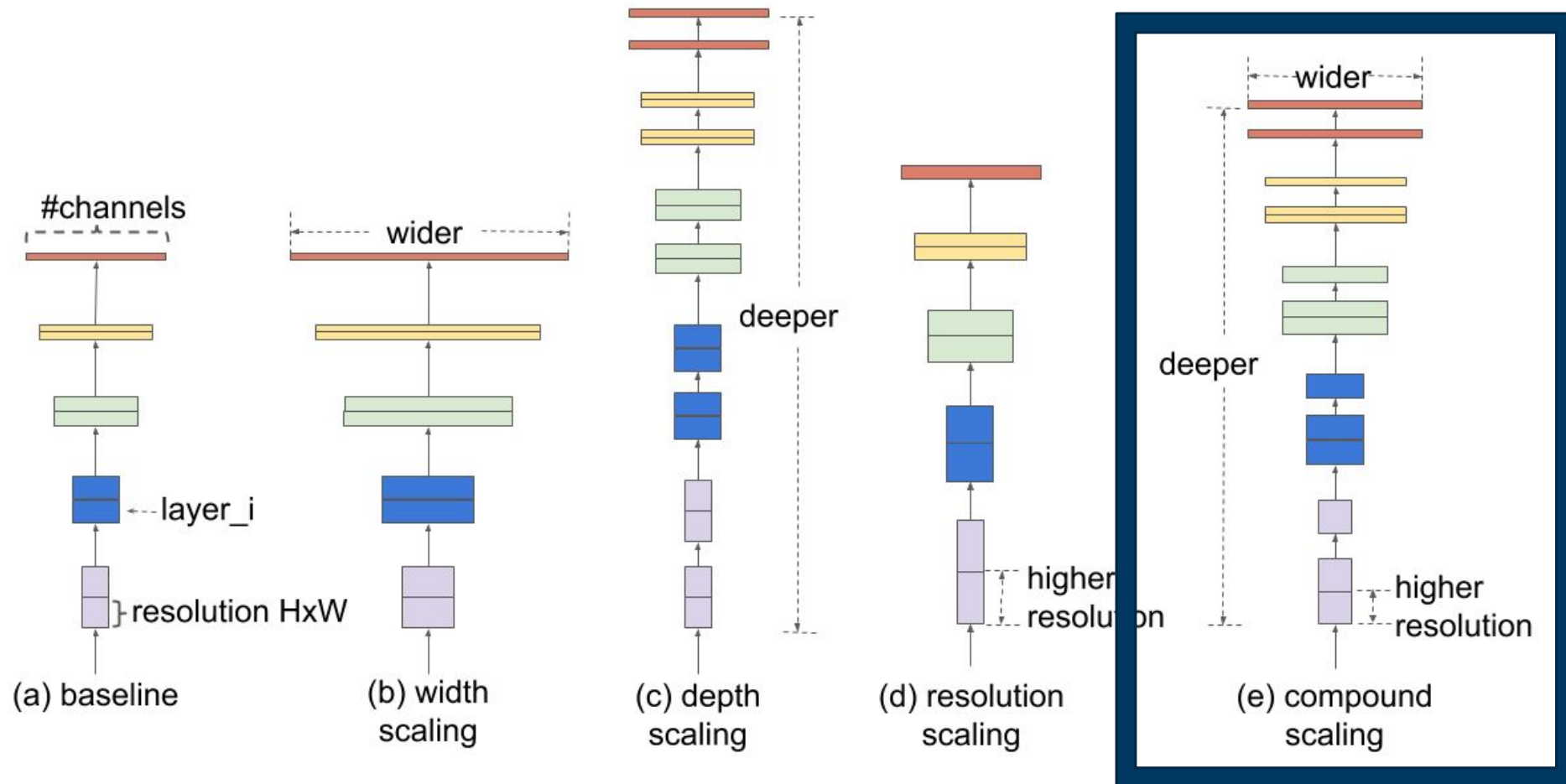
ResNet: Introduces residual connections. Improve gradient flow
-> Can train deeper networks!

Not only accuracy is important! Efficiency also matters.

- MobileNet: Make networks more efficient in inference to run on edge devices / smartphone.
 - Uses depth-wise separable convolutions.
- **Other aspect of efficiency: If I have more computational resources for training / inference, how do I best use it?**
 - EfficientNet



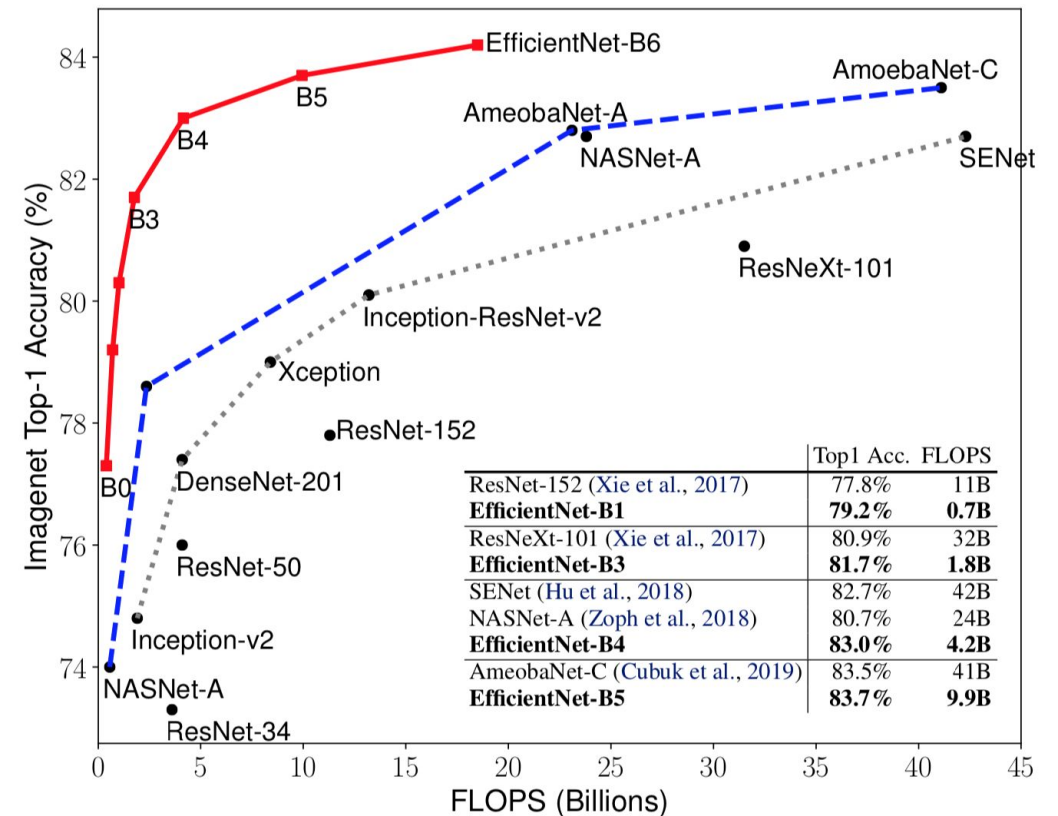
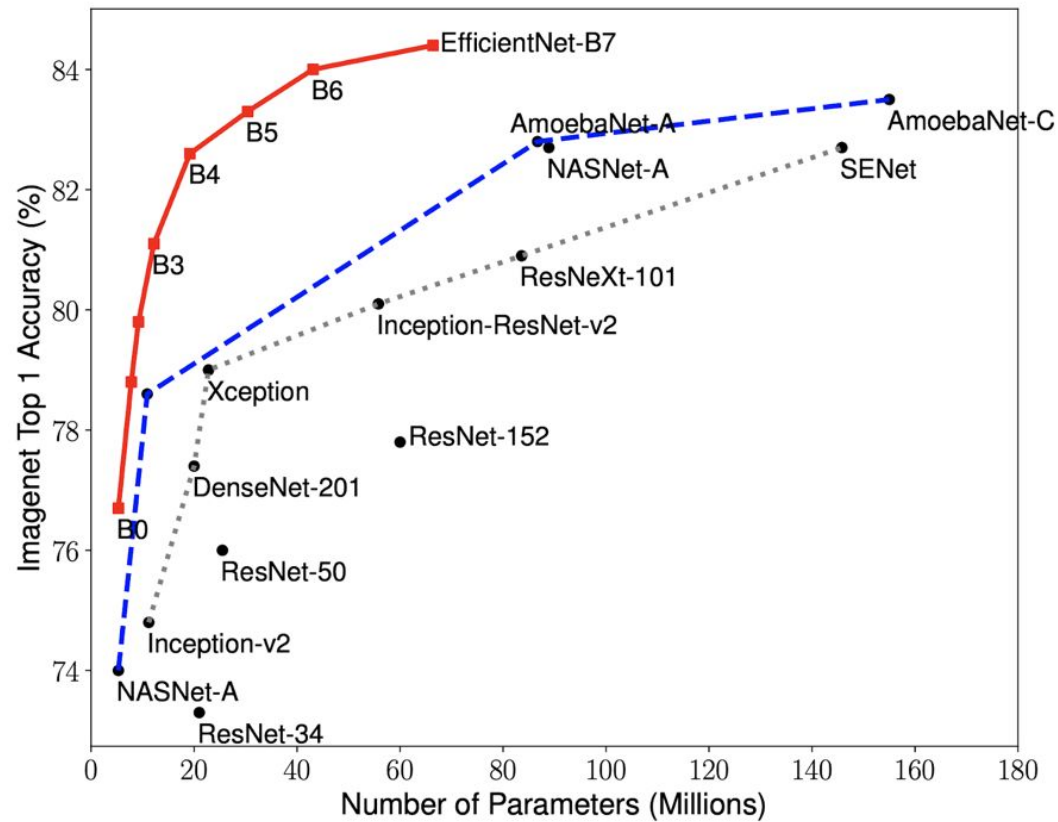
EfficientNet (2019)



EfficientNet

Tan & Le, *ICML* 2019

EfficientNet (2019)



Tan & Le, *ICML* 2019

<https://ai.googleblog.com/2019/05/efficientnet-improving-accuracy-and.html>

Neural Architecture Search

EfficientNet is a simple version of *Neural Architecture Search*:

Trying to find an optimal architecture for a given compute budget

(= best accuracy / performance @ given # Train / Inference Operations)

EfficientNet: Fixed base architecture, three search dimensions (depth, width resolution)

Neural Architecture Search

EfficientNet is a simple version of *Neural Architecture Search*:

Trying to find an optimal architecture for a given compute budget

(= best accuracy / performance @ given # Train / Inference Operations)

EfficientNet: Fixed base architecture, three search dimensions (depth, width resolution)

Can we find better architectures with less constraints? (Don't specify base architecture?)

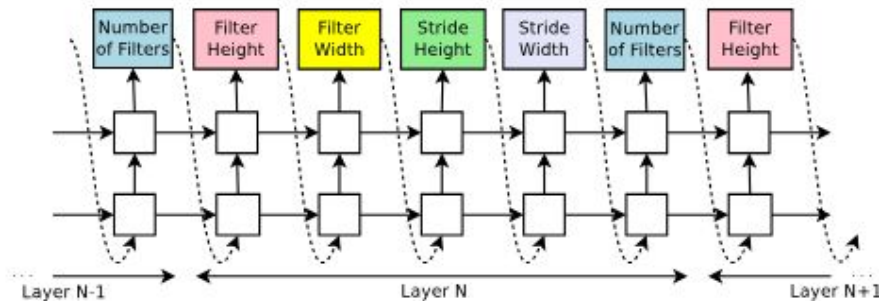
Need better search heuristics!

Neural Architecture Search

<https://arxiv.org/abs/1611.01578>

“Neural Architecture Search with Reinforcement Learning”

- Recurrent Neural Network (RNN) generates convolutional architecture descriptions



- Trained via Reinforcement Learning (Reward: Validation Set Accuracy)
 - Results matched state-of-the-art image recognition architectures at the time

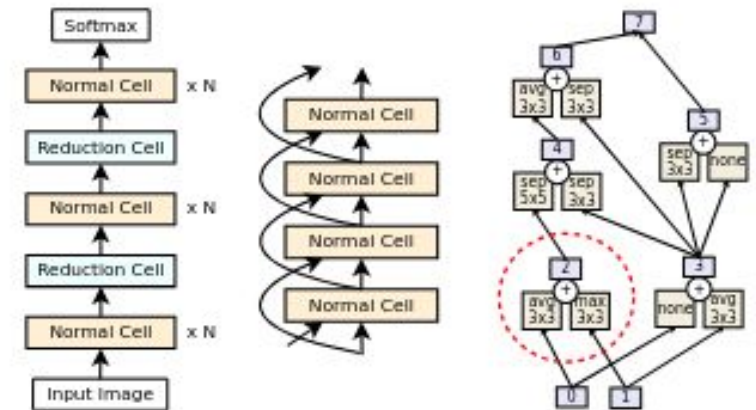
Neural Architecture Search

<https://arxiv.org/abs/1802.01548>

“Regularized Evolution for Image Classifier Architecture Search”

- Uses evolutionary algorithms to find good image recognition architectures:
 - Defines search space (= architecture parametrization)
 - Different architectures (= samples from search space) compete (via val set accuracies), winning architectures are combined for next round of competition

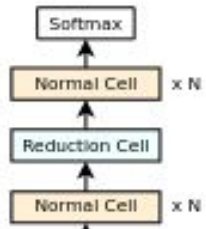
“Search Space”



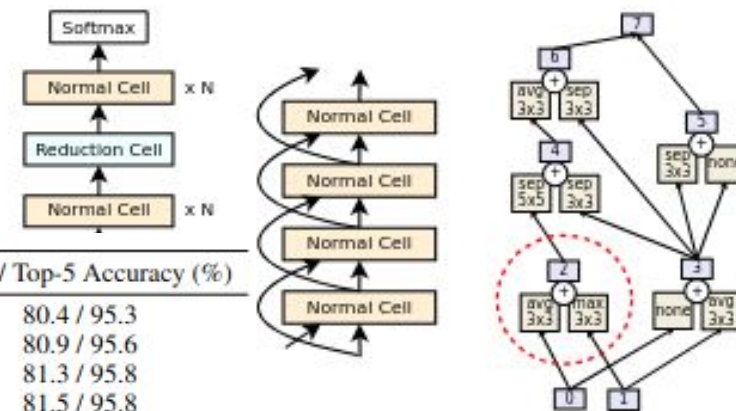
Neural Architecture Search

<https://arxiv.org/abs/1802.01548>

“Regularized Evolution for Image Classifier Architecture Search”

- Uses evolutionary algorithms to find good image recognition architectures:
 - Defines search space (= architecture parametrization)
 - Different architectures (= samples from search space) compete (via val set accuracies), winning architectures are combined for next round of competition
 - Results (slightly) beat state-of-the-art image recognition networks at the time
- 
- ```
graph BT; Input(()) --> N1[Normal Cell x N]; N1 --> R[Reduction Cell]; R --> N2[Normal Cell x N]; N2 --> S[Softmax];
```

## “Search Space”



| Model                     | # Parameters | # Multiply-Adds | Top-1 / Top-5 Accuracy (%) |
|---------------------------|--------------|-----------------|----------------------------|
| Incep-ResNet V2 [44]      | 55.8M        | 13.2B           | 80.4 / 95.3                |
| ResNeXt-101 [48]          | 83.6M        | 31.5B           | 80.9 / 95.6                |
| PolyNet [51]              | 92.0M        | 34.7B           | 81.3 / 95.8                |
| Dual-Path-Net-131 [7]     | 79.5M        | 32.0B           | 81.5 / 95.8                |
| GeNet-2 [47]*             | 156M         | –               | 72.1 / 90.4                |
| Block-QNN-B [52]*         | –            | –               | 75.7 / 92.6                |
| Hierarchical [30]*        | 64M          | –               | 79.7 / 94.8                |
| NASNet-A [54]             | 88.9M        | 23.8B           | 82.7 / 96.2                |
| PNASNet-5 [29]            | 86.1M        | 25.0B           | 82.9 / 96.2                |
| AmoebaNet-A (N=6, F=190)* | 86.7M        | 23.1B           | 82.8 / 96.1                |
| AmoebaNet-A (N=6, F=448)* | 469M         | 104B            | 83.9 / 96.6                |



# Neural Architecture Search

<https://arxiv.org/abs/1802.01548>

“Regularized Evolution for Image Classifier Architecture Search”

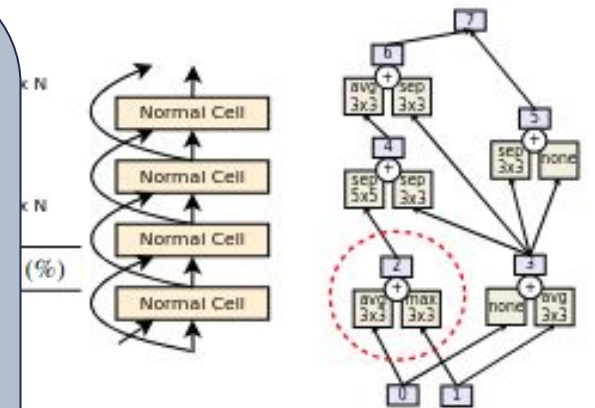
- Uses evolutionary algorithms to find good image recognition architectures:
  - Defines search space (= architecture parametrization)
  - Different architectures (= samples from search space)

BUT: Neural architecture search is very compute intensive and typically only brings minor improvements over hand-designed architectures improved by “normal” scaling (e.g. EfficientNet)

Later work also applies it to transformers, e.g.

<https://arxiv.org/abs/2107.00651>

“Search Space”



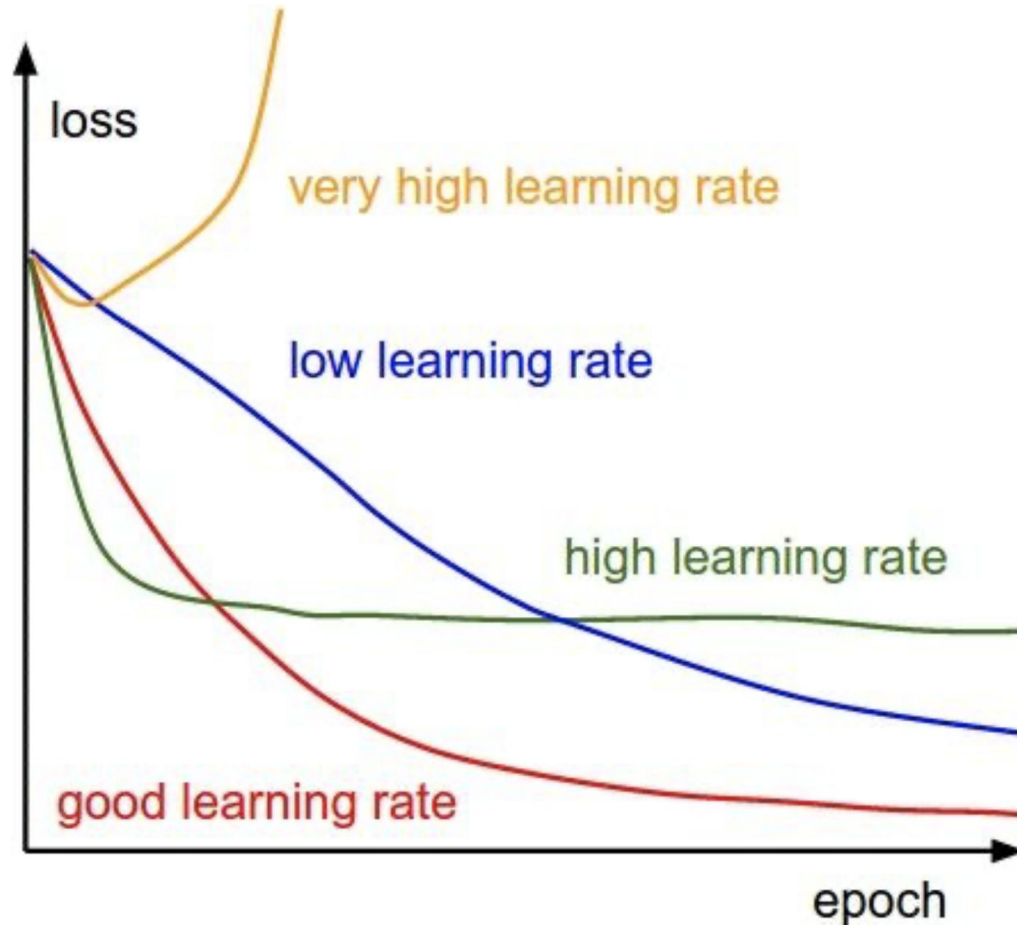
AmoebaNet-A (N=6, F=448)\*      469M      104B      85.9 / 96.6

# Training large-scale CNNs

---

LEARNING RATE SCHEDULES, MODEL ENSEMBLING & DATA AUGMENTATION

# Choosing the learning rate



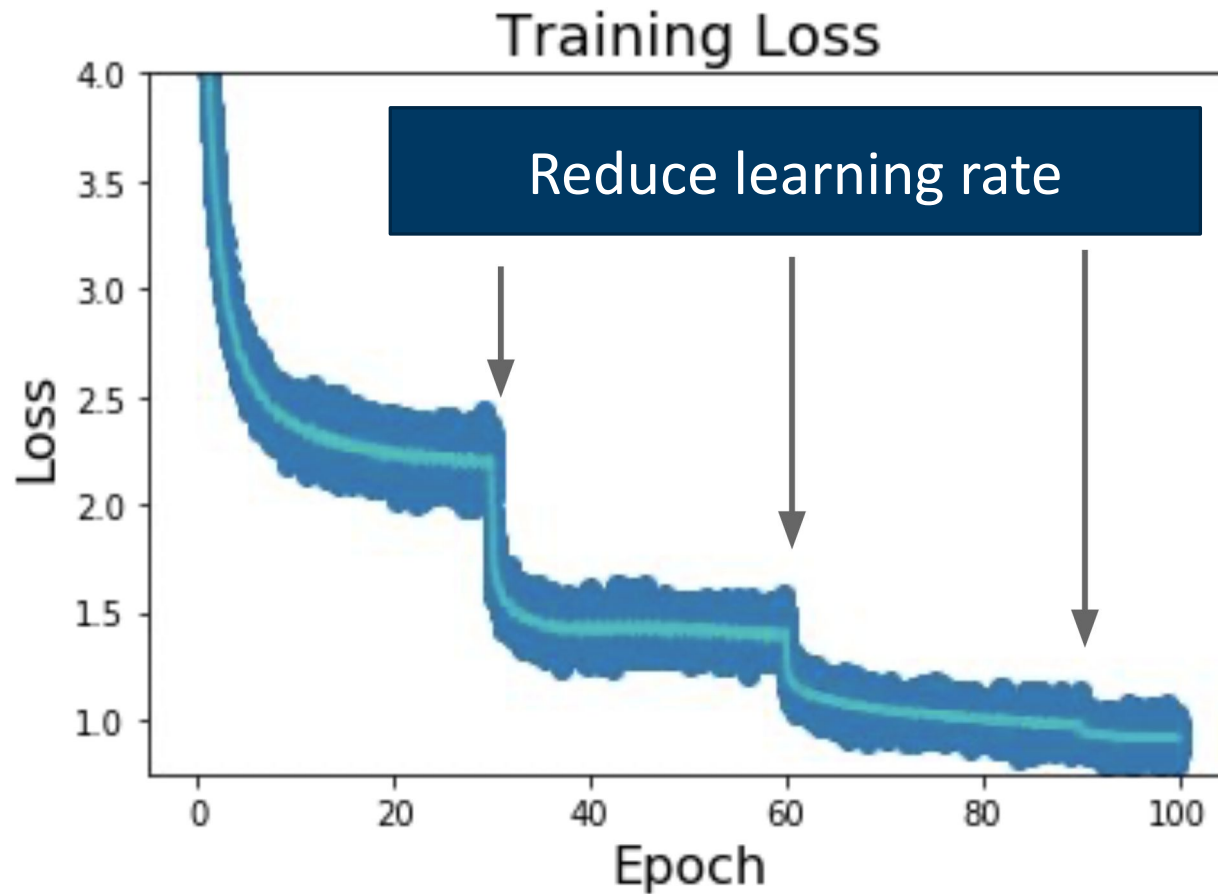
Which of these learning rates should we use?

A combination:

- Start with a high learning rate
- Decrease over time

-> Learning rate schedule

# Learning rate decay



## Step decay

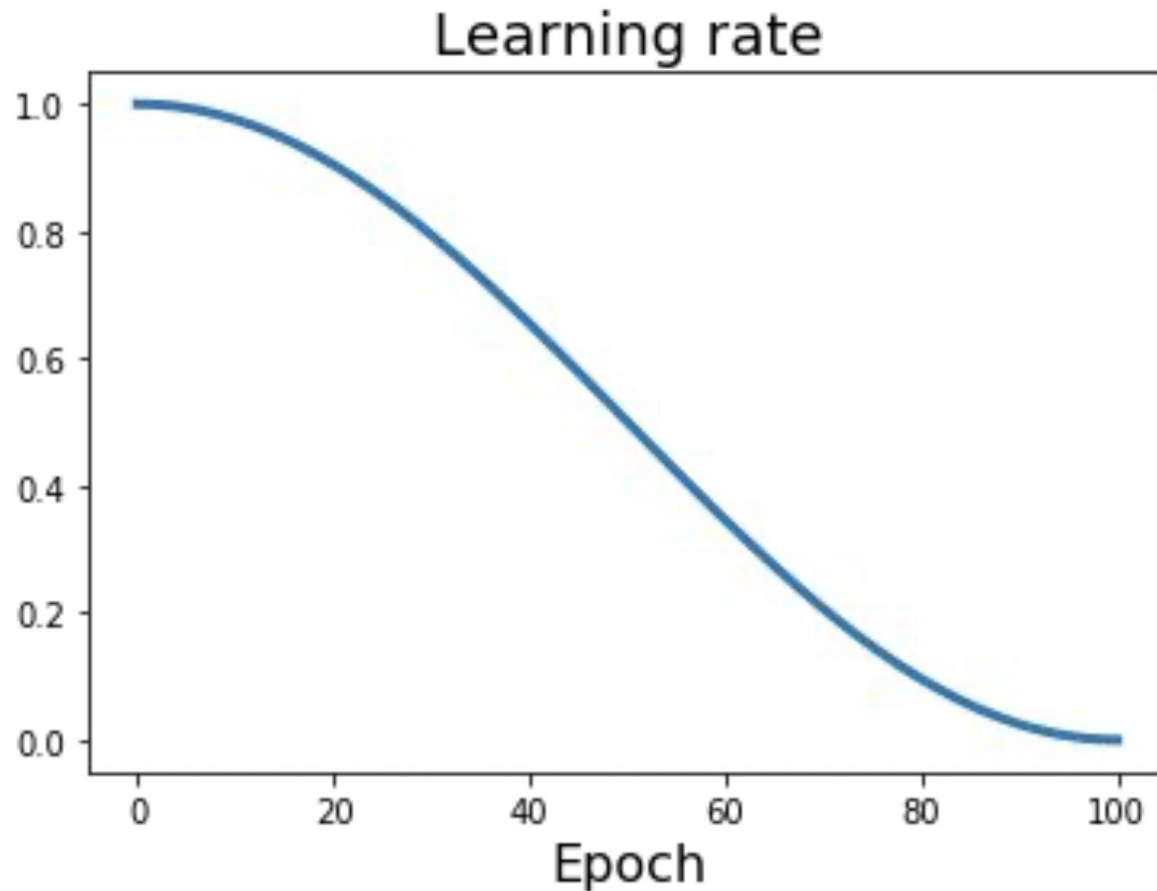
Reduce learning rate by a fixed factor

Example ResNet (left):

- Multiply LR by 0.1
- After 30, 60, 90 epochs

# Cosine learning rate decay

---



Learning rate  $\eta_t$  for epoch  $t$

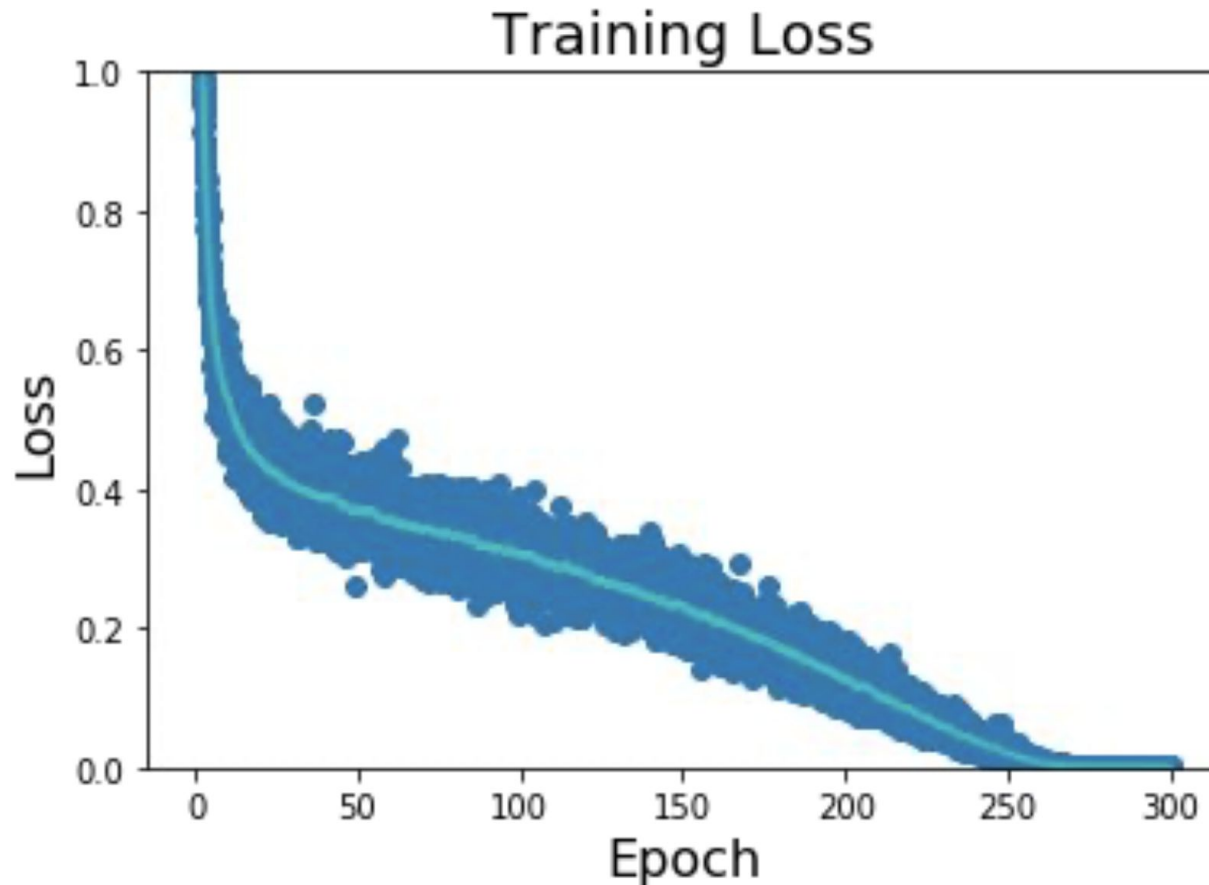
$$\eta_t = \frac{1}{2}\eta_0 [1 + \cos(t\pi/T)]$$

$\eta_0$ : initial learning rate

$T$ : total number of epochs

# Cosine learning rate decay

---



Learning rate  $\eta_t$  for epoch  $t$

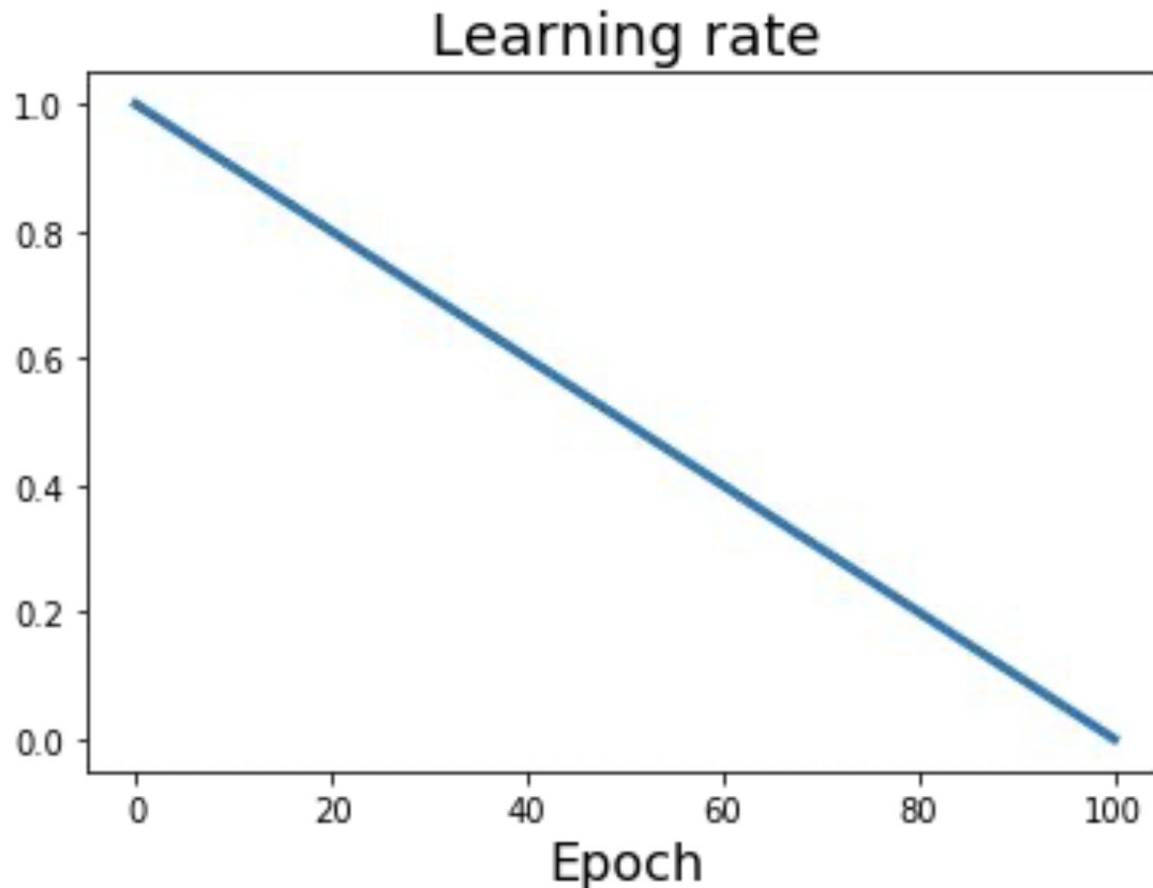
$$\eta_t = \frac{1}{2}\eta_0 [1 + \cos(t\pi/T)]$$

$\eta_0$ : initial learning rate

$T$ : total number of epochs

# Linear learning rate decay

---



Learning rate  $\eta_t$  for epoch  $t$

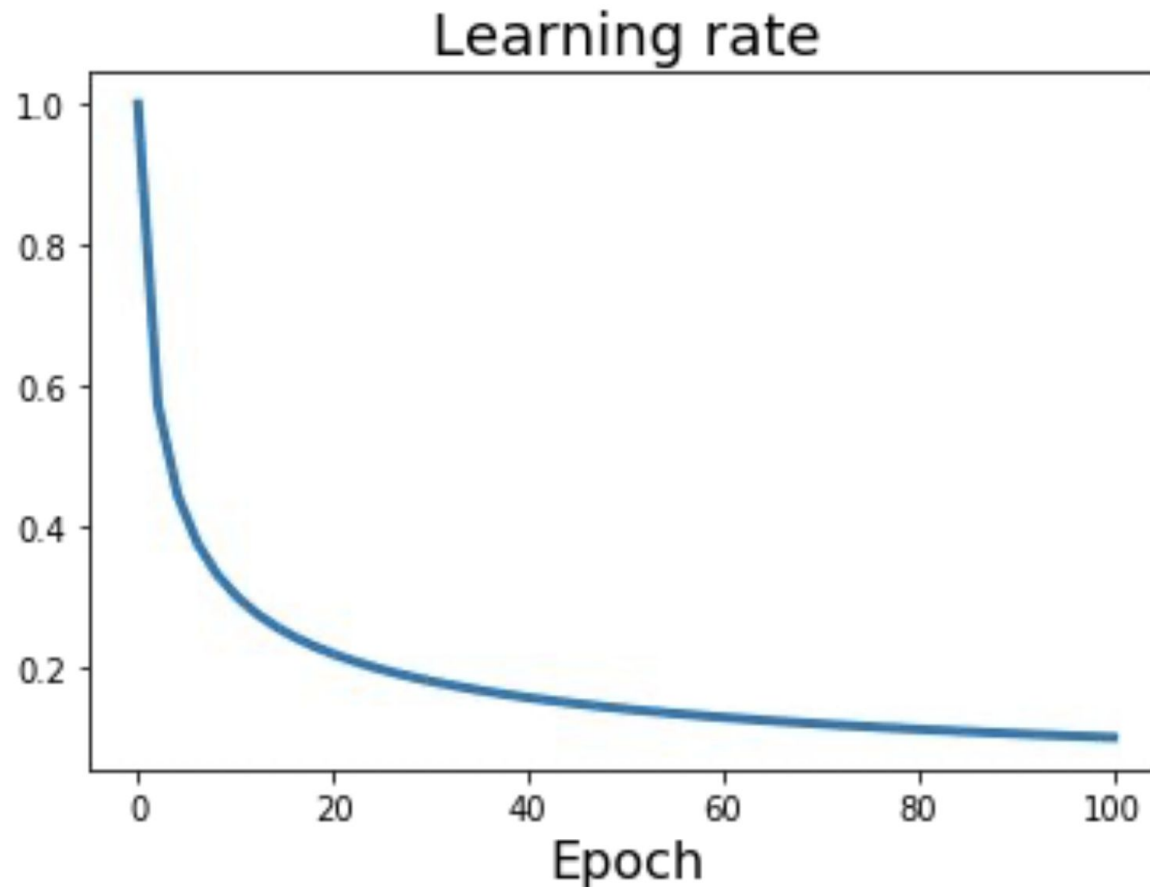
$$\eta_t = \eta_0 [1 - t/T]$$

$\eta_0$ : initial learning rate

$T$ : total number of epochs

# Inverse square root learning rate decay

---



Learning rate  $\eta_t$  for epoch  $t$

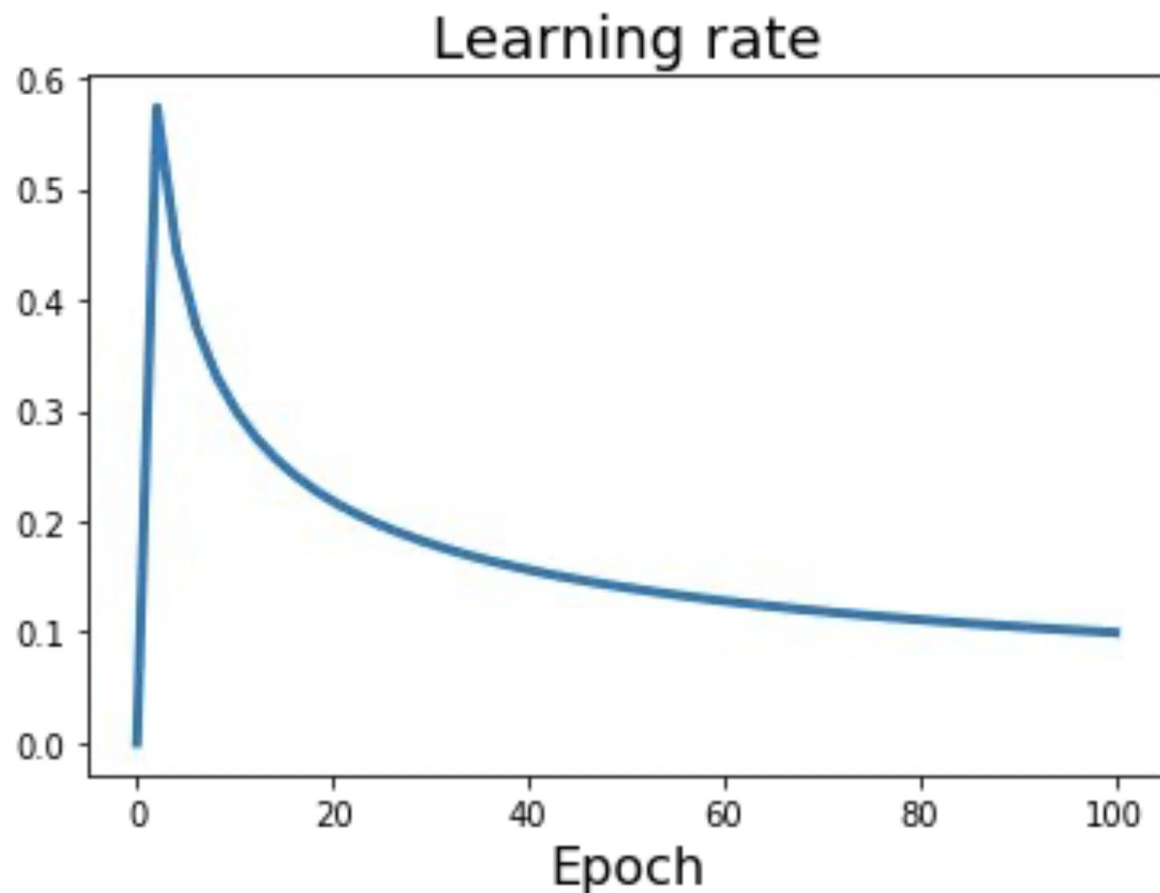
$$\eta_t = \eta_0 / \sqrt{t}$$

$\eta_0$ : initial learning rate



# Linear warmup

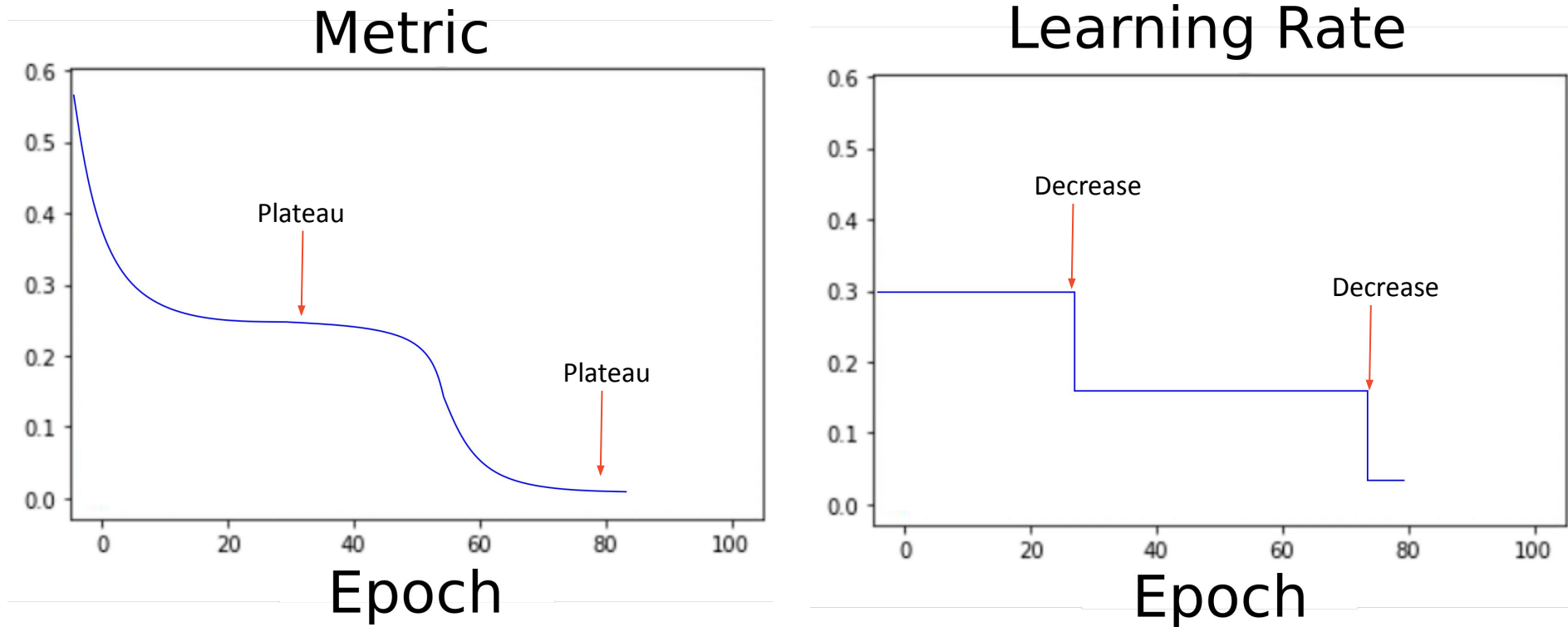
---



High initial learning rate can make loss explode

- Linear warmup over a few epochs

# Metric dependent learning rate



Reduce learning rate when metric (e.g. classification accuracy) plateaus

# Empirical rule of thumb

---

Recall: Stochastic gradient descent approximates true gradient by sampling minibatches

If samples are i.i.d., variance of gradient estimate decreases with  $1/N$

Multiply learning rate by  $N$  when you increase the batch size by a factor of  $N$

More in depth analysis of scaling the learning rate with batch size:

<https://stackoverflow.com/questions/53033556/how-should-the-learning-rate-change-as-the-batch-size-change>

<https://www.jmlr.org/papers/volume23/20-1258/20-1258.pdf>

# Model ensembling

---

# Model ensembling

---

1. Train multiple independent models
2. Average predictions at test time; several options:
  - Majority vote
  - Average probabilities, then pick maximum

Enjoy an extra 2% performance

# Model ensembling

---

1. Train multiple independent models
2. Average predictions at test time; several options:
  - Majority vote
  - Average probabilities, then pick maximum

Enjoy an extra 2% performance

Random initialization and many local minima in loss landscape  
-> each training run will result in a different model

# Model ensembling

---

1. Train multiple independent models
2. Average predictions at test time; several options:
  - Majority vote
  - Average probabilities, then pick maximum

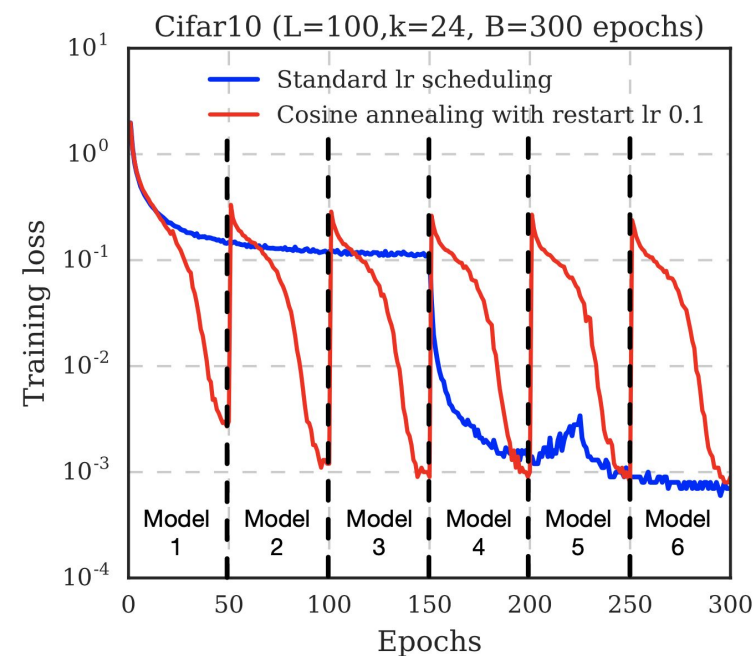
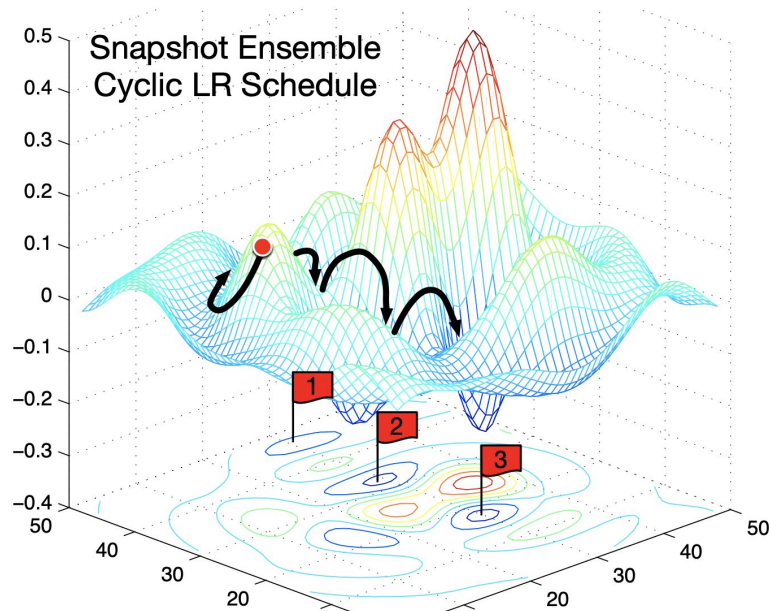
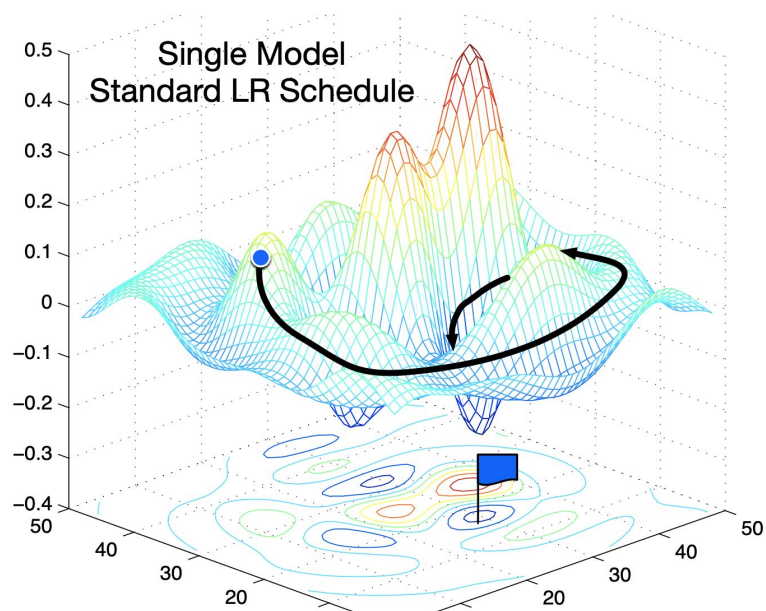
Enjoy an extra 2% performance

**BUT:** higher computational cost during training (train several models) and prediction (predict several models)

Random initialization and many local minima in loss landscape  
-> each training run will result in a different model

# Model ensembling: tips and tricks

Instead of training  $N$  independent models, use multiple snapshots of a single model during training



Works well in combination with cyclic learning rates



“Momentum” for model weights.

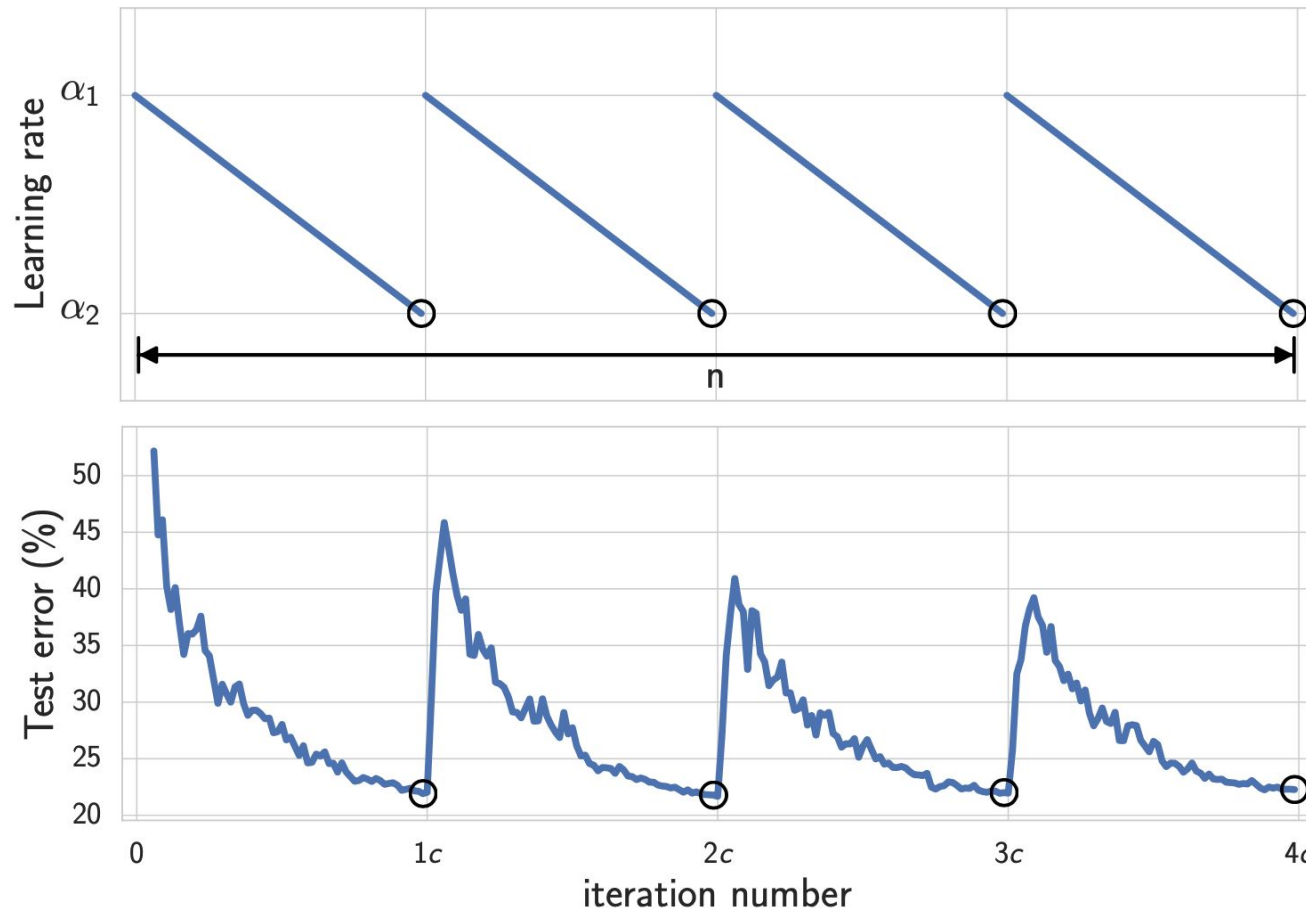
# Polyak averaging

Instead of averaging model predictions, average weights over time

```
1 while not converged:
2 x = dataset.next()
3 loss = model.forward(x)
4 dw = model.backward()
5 w += -learning_rate * dw
6 w_test = 0.995 * w_test + 0.005 * w
```

Use for testing

# Stochastic weight averaging



Combination of Polyak averaging and cyclic learning rates

- Average the weights over points of minimal LR

# Data augmentation

---

# Data augmentation

---

Increase training dataset by applying transformations (*augmentation*) to the data

# Data augmentation

---

Increase training dataset by applying transformations (*augmentation*) to the data

2 different motivations:

- Increase the effective data size
  - Use augmentations that stay on data manifold
  - e.g. for natural images: horizontal flips are ok, vertical flips not

# Data augmentation

---

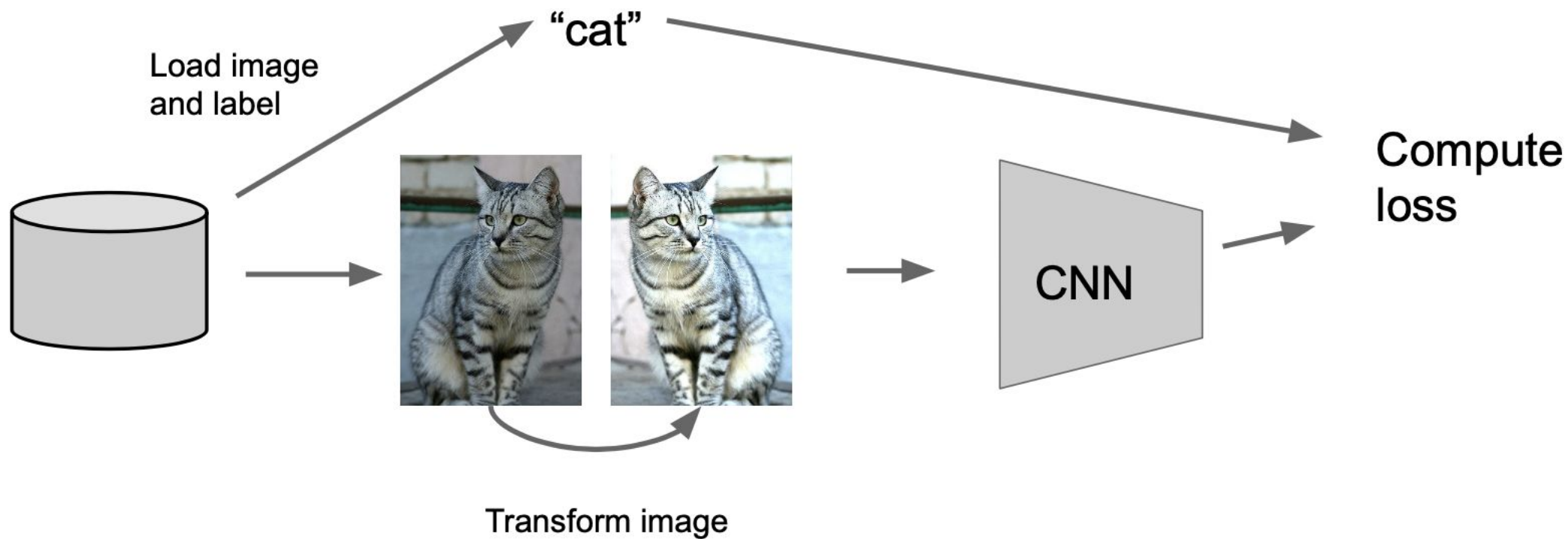
Increase training dataset by applying transformations (*augmentation*) to the data

2 different motivations:

- Increase the effective data size
  - Use augmentations that stay on data manifold
  - e.g. for natural images: horizontal flips are ok, vertical flips not
- Data artifacts: increase robustness of the model
  - Simulate image artifacts with augmentation (rare in training data)
  - Use augmentation that goes from “normal” to “artifact” manifold
  - e.g. images from smartphone snapshots: simulate blur and multiple orientations

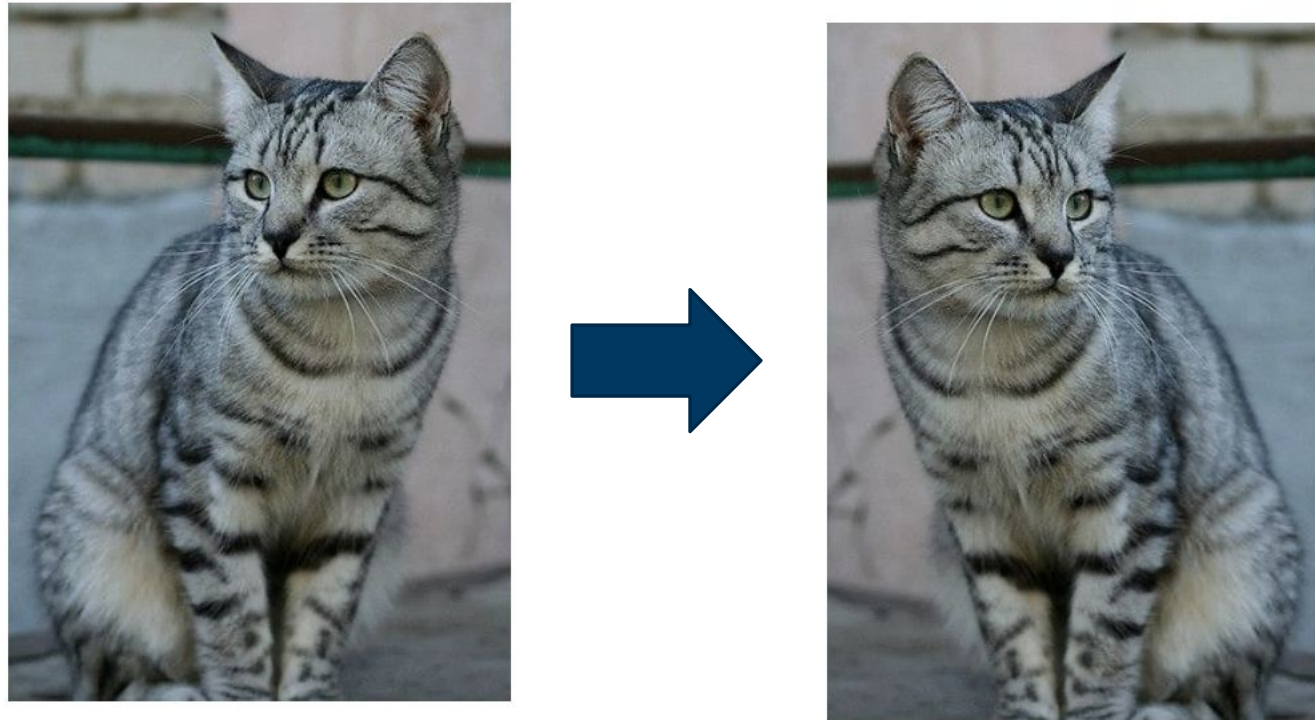
# Data augmentation

---



# Data augmentation: horizontal flips

---





# Data augmentation: crops and scale

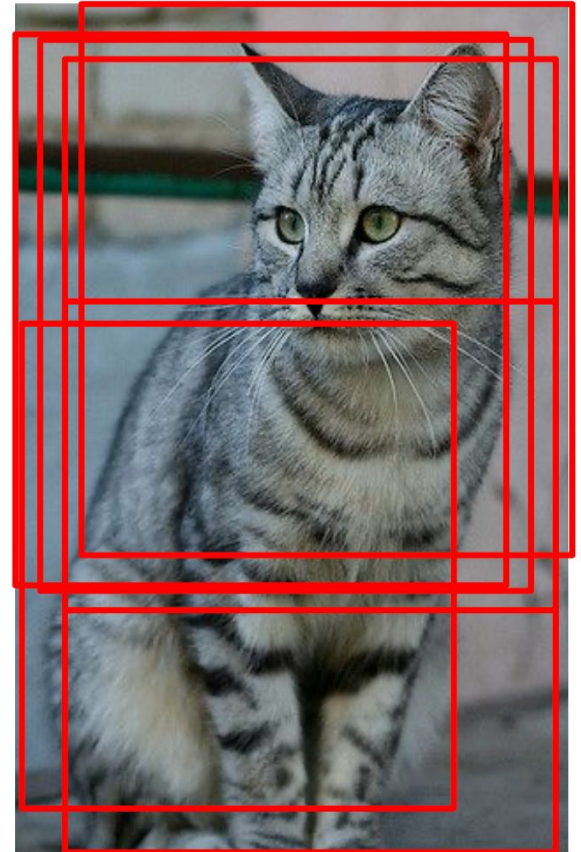
---

## Example: ResNet

### Training:

Sample random  
crops and scales

1. Pick random  $L \in [256, 480]$
  2. Resize training image such that short side has length  $L$
  3. Sample random  $224 \times 224$  patch
- 
1. Resize image to five scales  $\{224, 256, 384, 480, 640\}$
  2. For each size, 10 crops of  $224 \times 224$  (4 corners + centers)  $\times$  (2 flips)

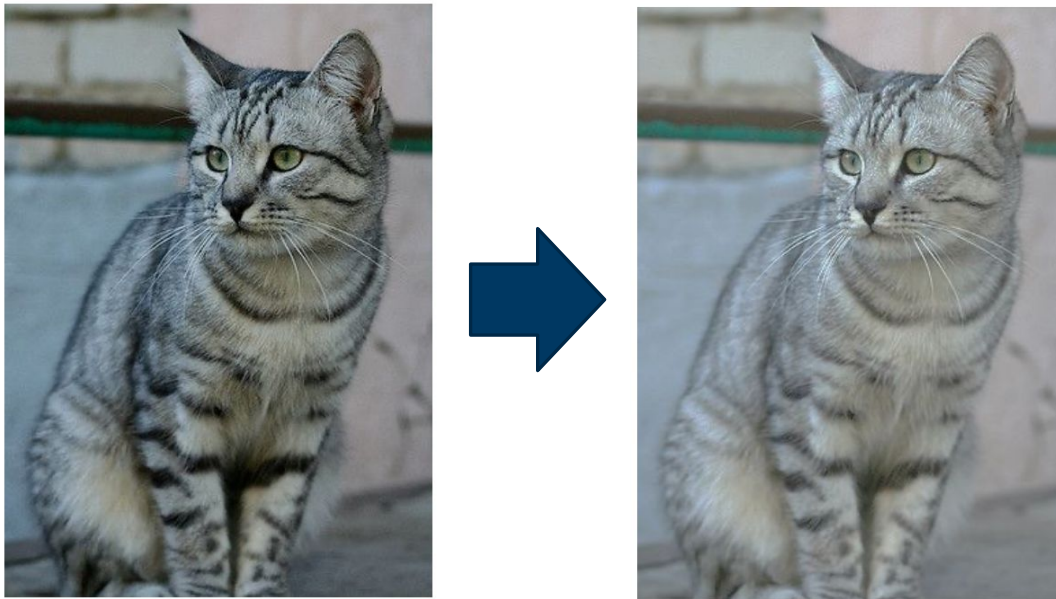


# Data augmentation: color jitter

---

## **Simplest version:**

Randomize brightness and contrast



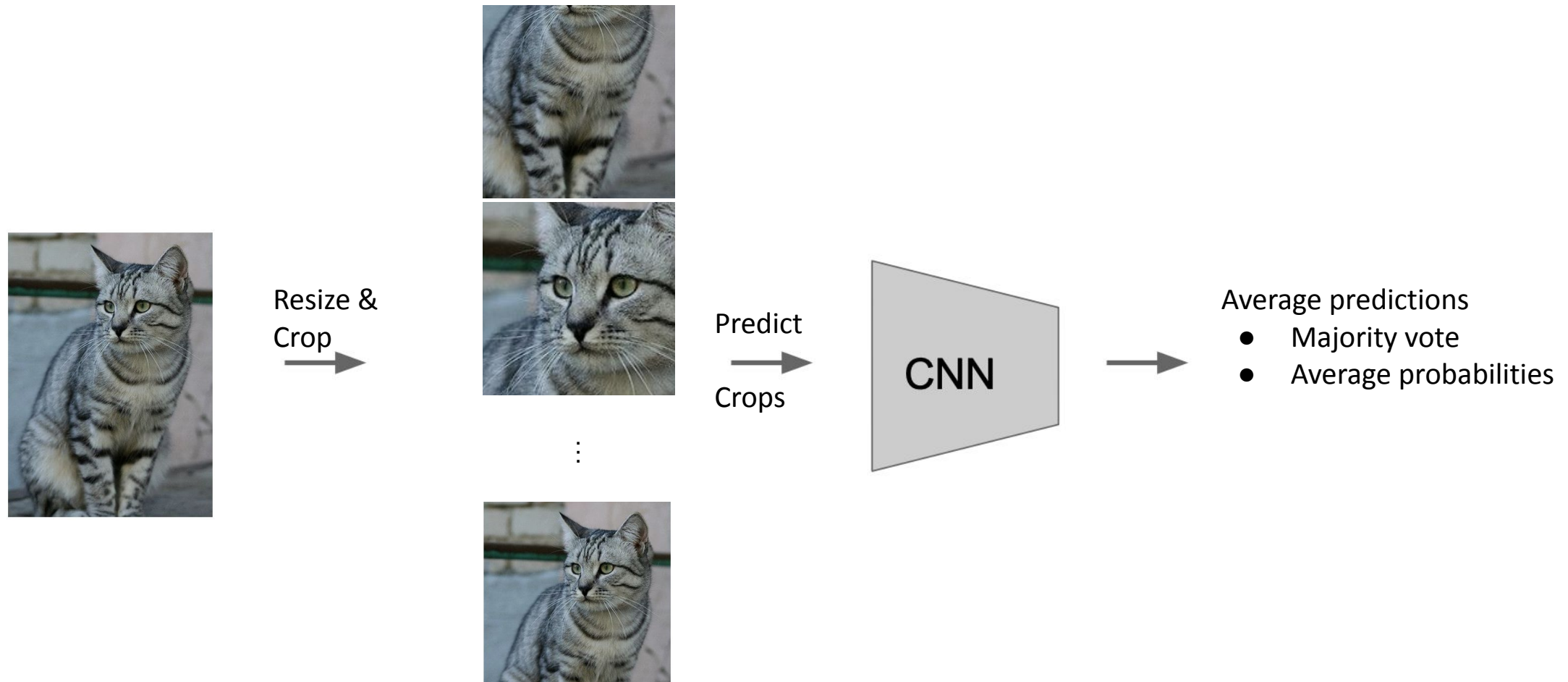
## **Advanced version:**

1. Compute PCA in (R,G,B) space across all pixels
2. Sample color offset along PCs
3. Add offset to all pixels in one image

Krizhevsky et al. "ImageNet Classification with Deep Convolutional Neural Networks" NeurIPS 2012  
Also in ResNets etc.

# Test-time data augmentation

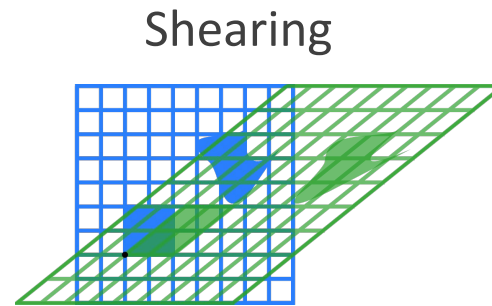
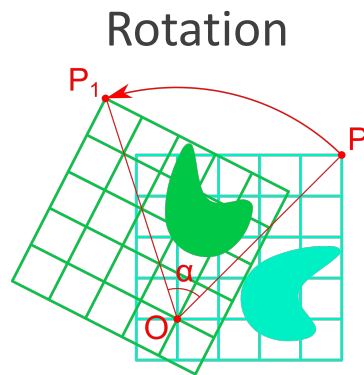
Apply several augmentations **during prediction** of new data point



# Data augmentation

There are many options. Get creative, depending on your problem

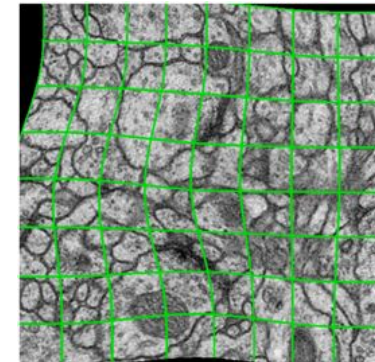
- Crops
- Scale
- Flips
- Color
- Noise



Lens distortion



Non-rigid deformations



...

# Advanced data augmentation

MixUp: Take linear combination of input and target for two training samples

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda)x_j, & \text{where } x_i, x_j \text{ are raw input vectors} \\ \tilde{y} &= \lambda y_i + (1 - \lambda)y_j, & \text{where } y_i, y_j \text{ are one-hot label encodings}\end{aligned}$$

“cat”



“dog”



$0.7 * \text{“cat”} + 0.3 * \text{“dog”}$





# Advanced data augmentation

---

MixUp: Take linear combination of input and target for two training samples

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda)x_j, & \text{where } x_i, x_j \text{ are raw input vectors} \\ \tilde{y} &= \lambda y_i + (1 - \lambda)y_j, & \text{where } y_i, y_j \text{ are one-hot label encodings}\end{aligned}$$

CutMix: Mix patches of the two inputs instead

- Target: linear combination weighted with patch ratio

ResNet-50



Mixup [47]



Cutout [3]



CutMix

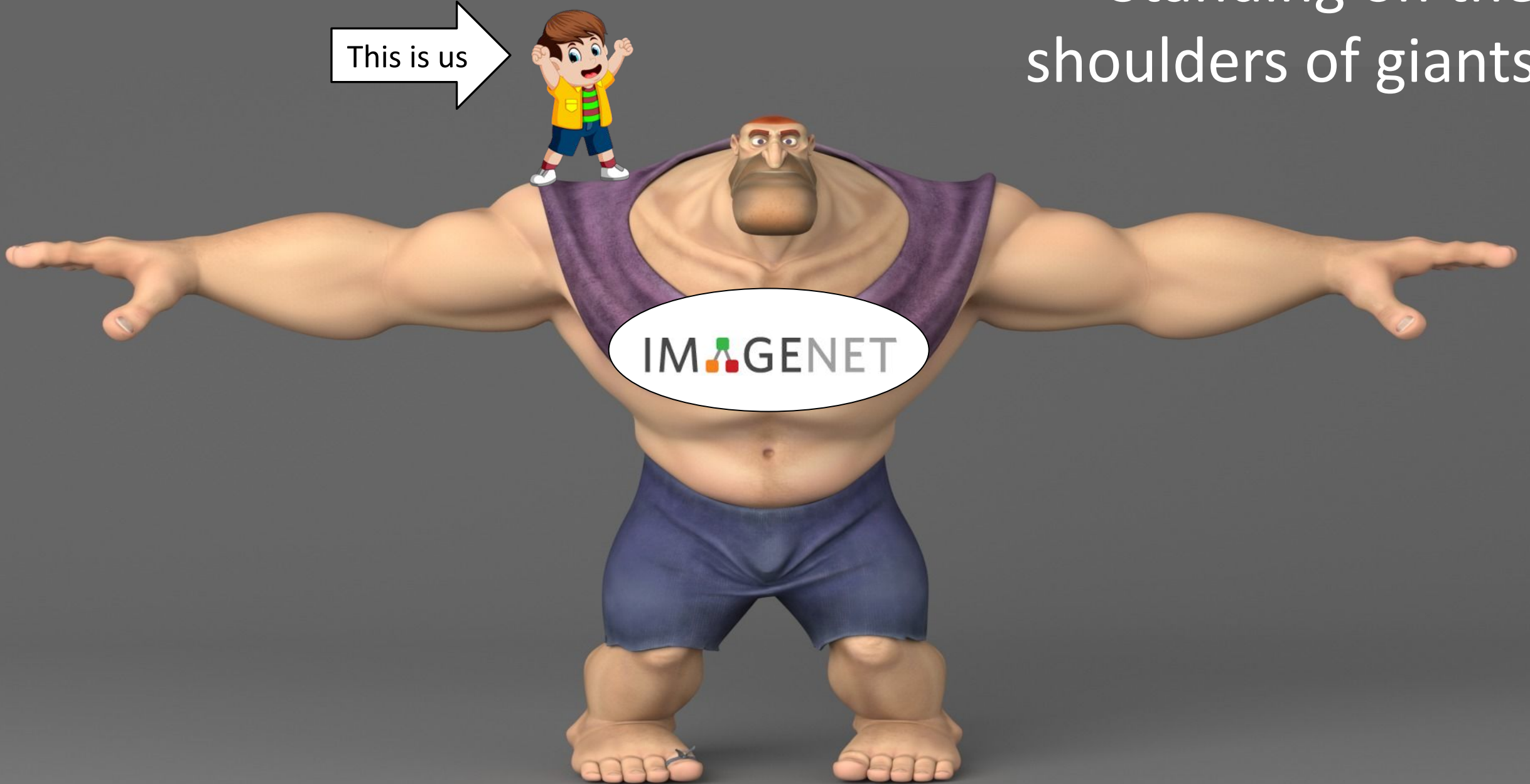


# Transfer learning

---

Standing on the  
shoulders of giants

This is us





# Transfer learning: motivation

Illustrative example

Suppose you want to build an app to recognize flowers...



alpine sea holly



buttercup



fire lily



cape flower



anthurium



californian poppy



foxglove



artichoke



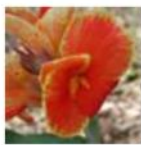
frangipani



cauleya spicata



azalea



canna lily



fritillary



clematis

Training a convolutional net from scratch won't work well

1000 images  
100 classes

# Transfer learning: the idea

1

## Pre-training

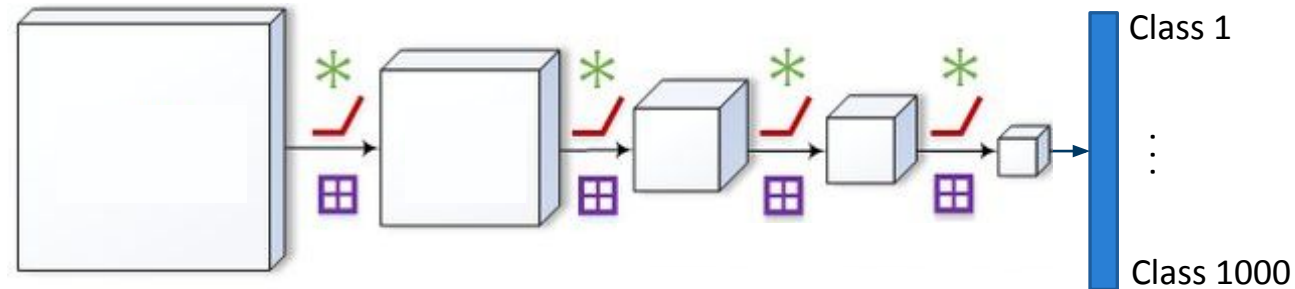
(massive data)

IMAGENET

1.2 million

images

1000 object  
classes

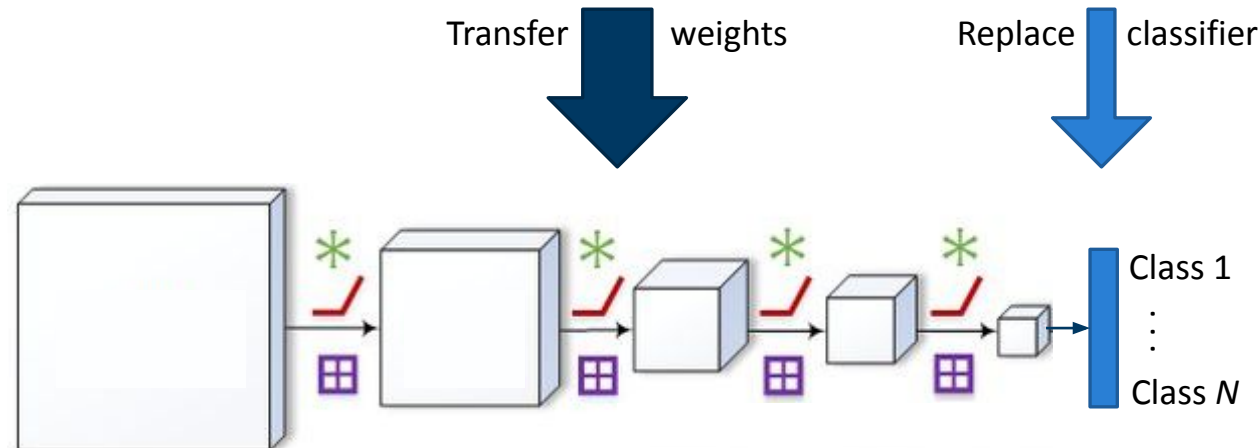


IMAGENET

2

## Fine-tuning

(much less data)



Recognize flowers

# Transfer learning: the idea

1

## Pre-training

(massive data)

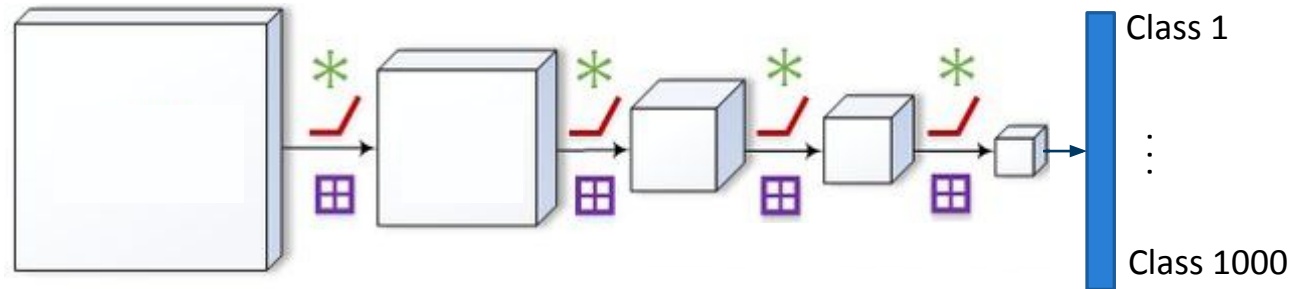
IMAGENET

1.2 million

images

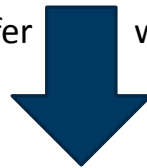
1000 object

classes



IMAGENET

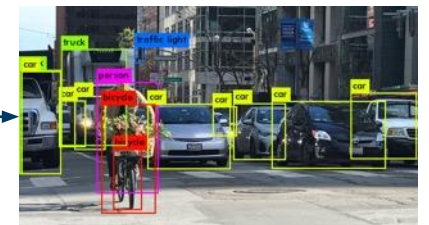
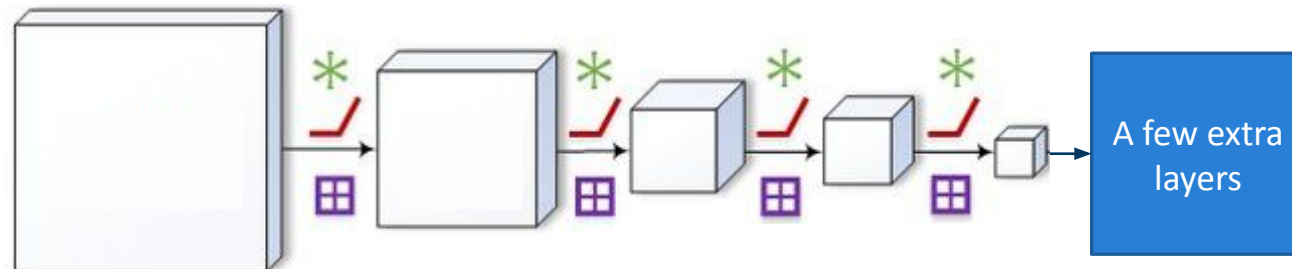
Transfer weights



2

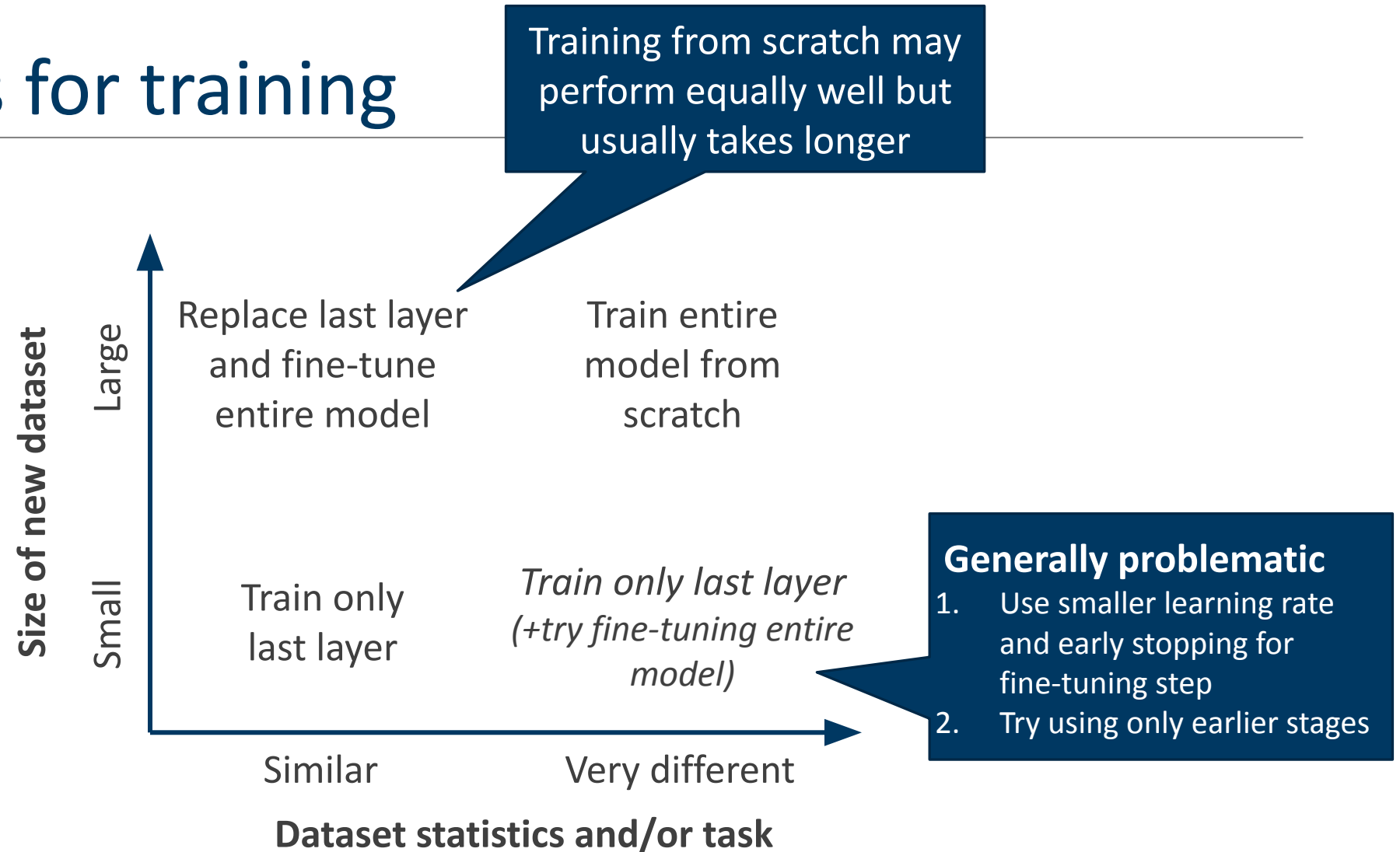
## Fine-tuning

(much less data)



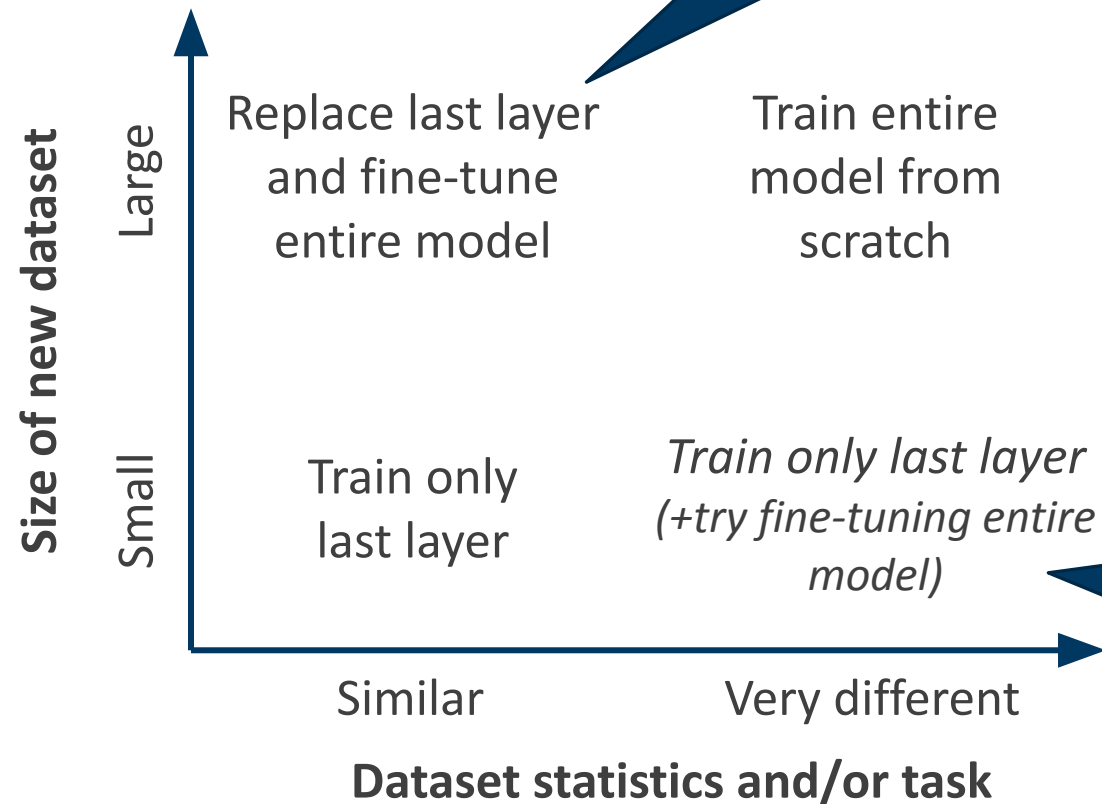
Object detection

# Strategies for training



# Strategies for training

Training from scratch may perform equally well but usually takes longer



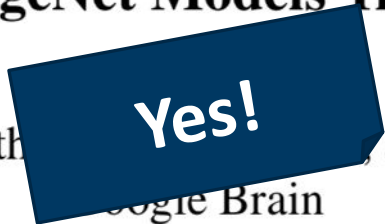
## Generally problematic

1. Use smaller learning rate and early stopping for fine-tuning step
2. Try using only earlier stages

“Simple strategies”

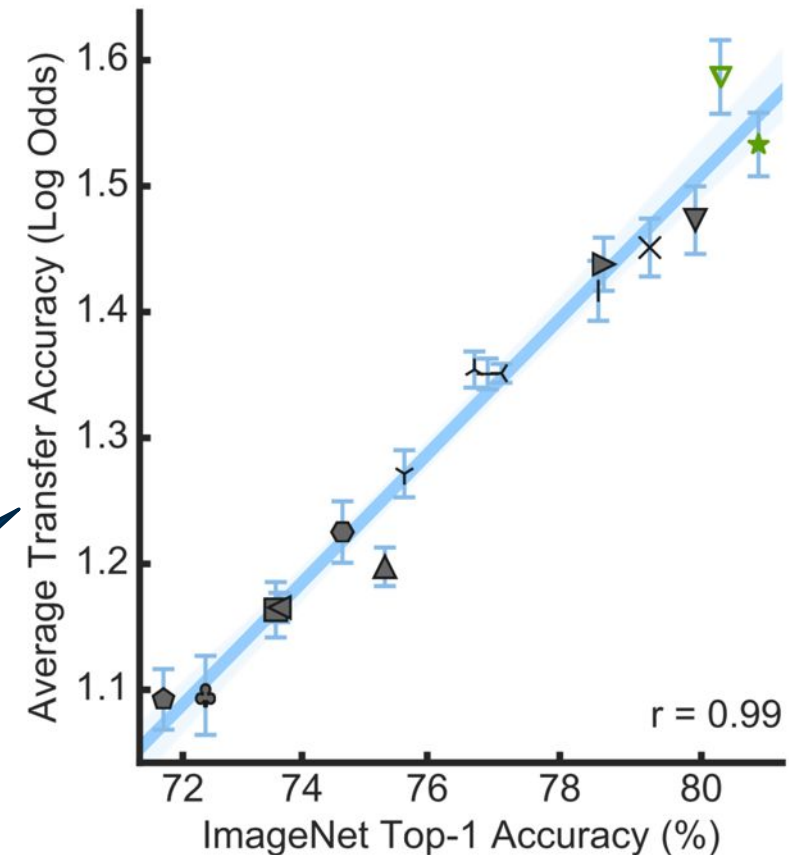
# Better ImageNet models lead to improvements on downstream tasks

## Do Better ImageNet Models Transfer Better?

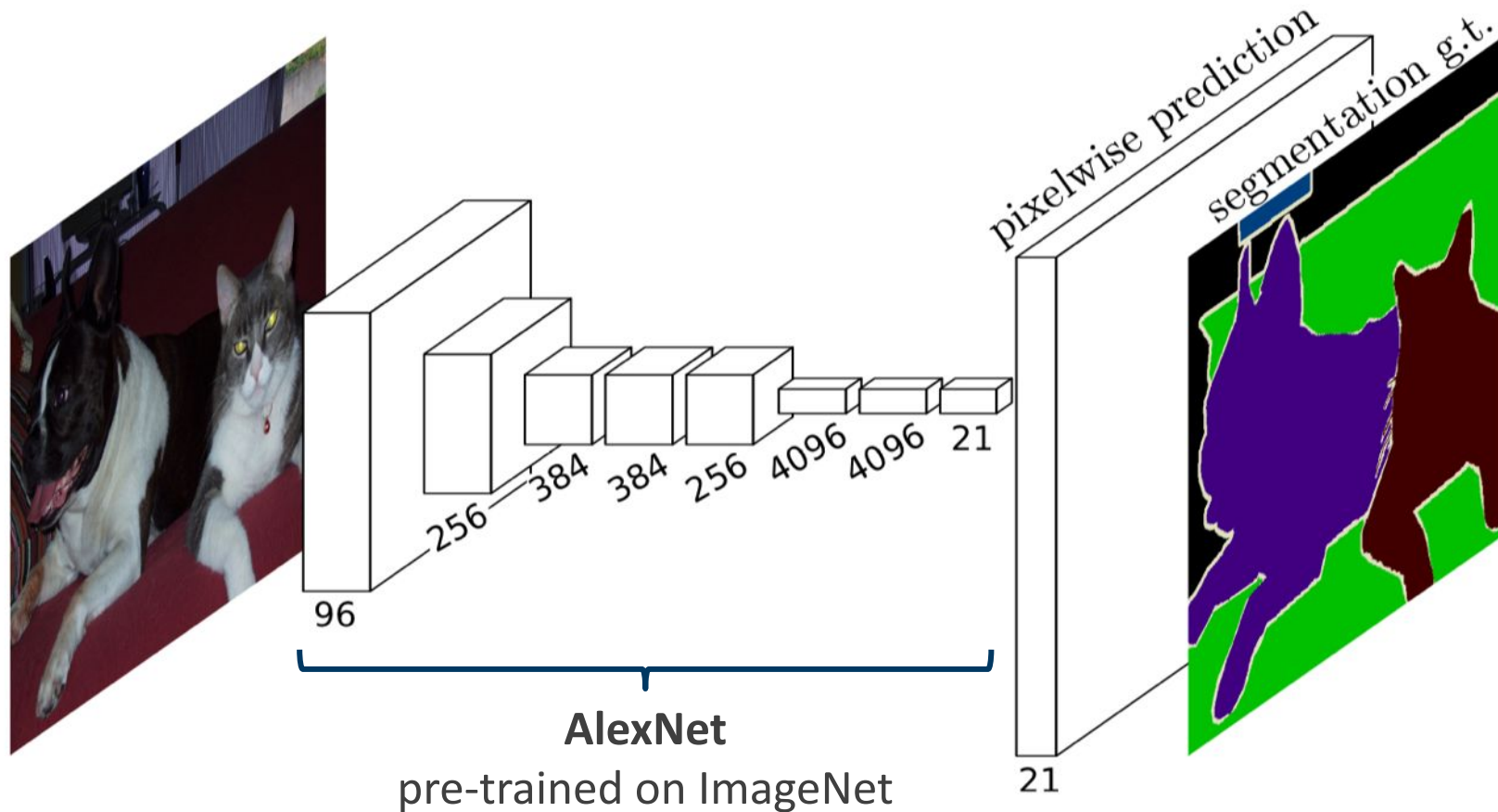
Simon Kornblith,  and Quoc V. Le  
{skornblith, shlens, qvl}@google.com

**Yes!**

Tested on 12 different image  
classification sets



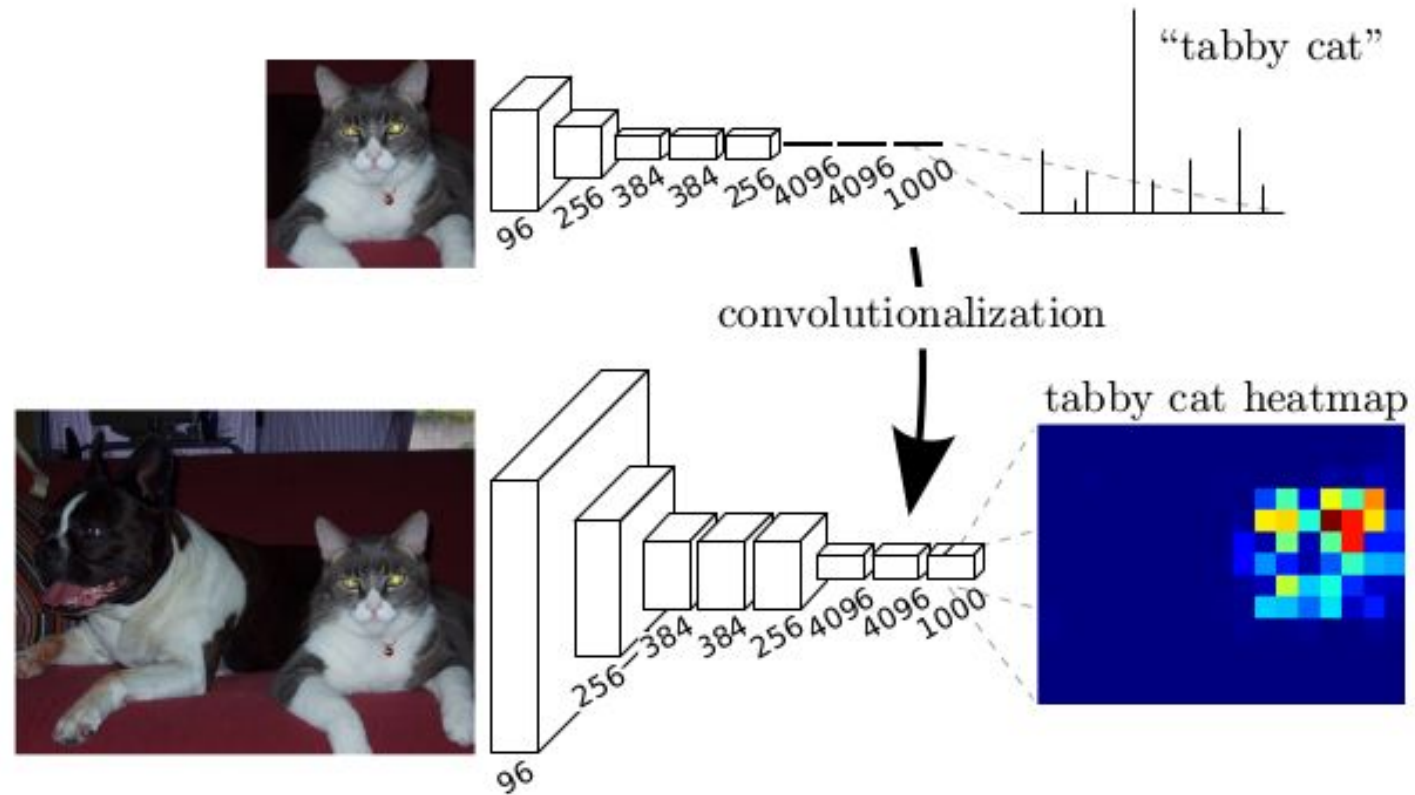
# Transfer learning for semantic segmentation





Will cover in more detail next lecture!

# Transfer learning for semantic segmentation





# Pre-training on ImageNet is everywhere ...

Human pose estimation

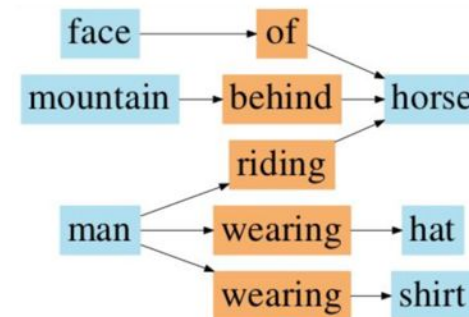
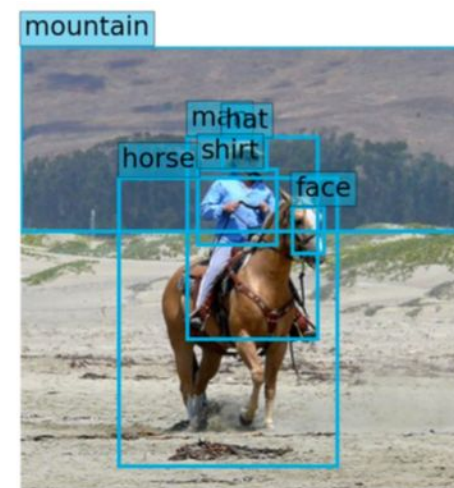


Image captioning



"two young girls are playing with  
lego toy."

Scene graph prediction

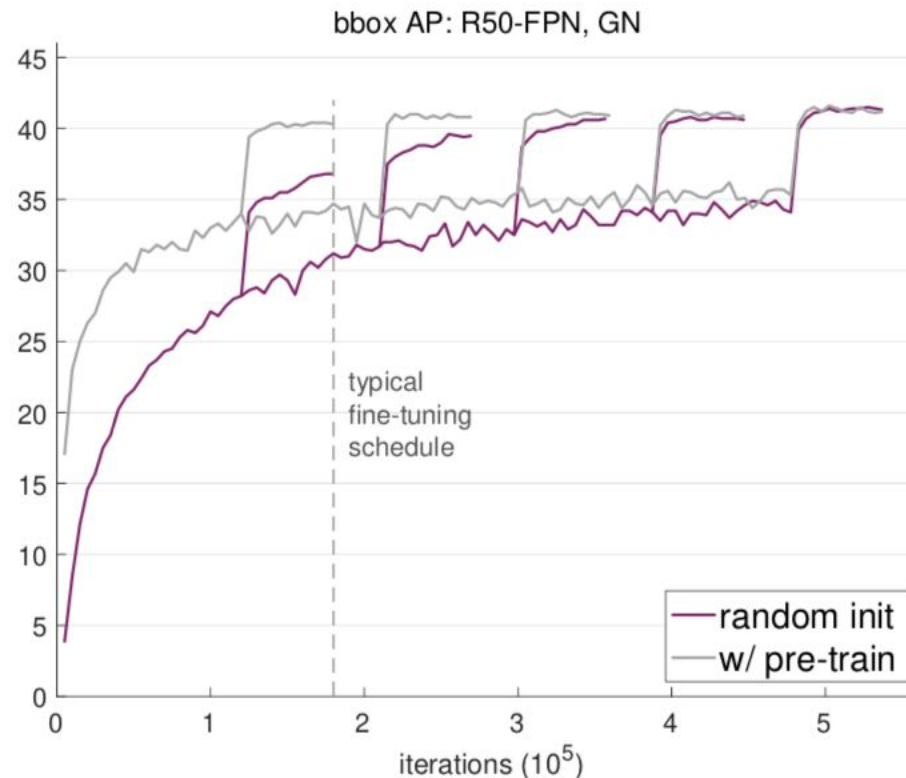


...

Many  
,  
many  
more

# Transfer learning from ImageNet is the standard...

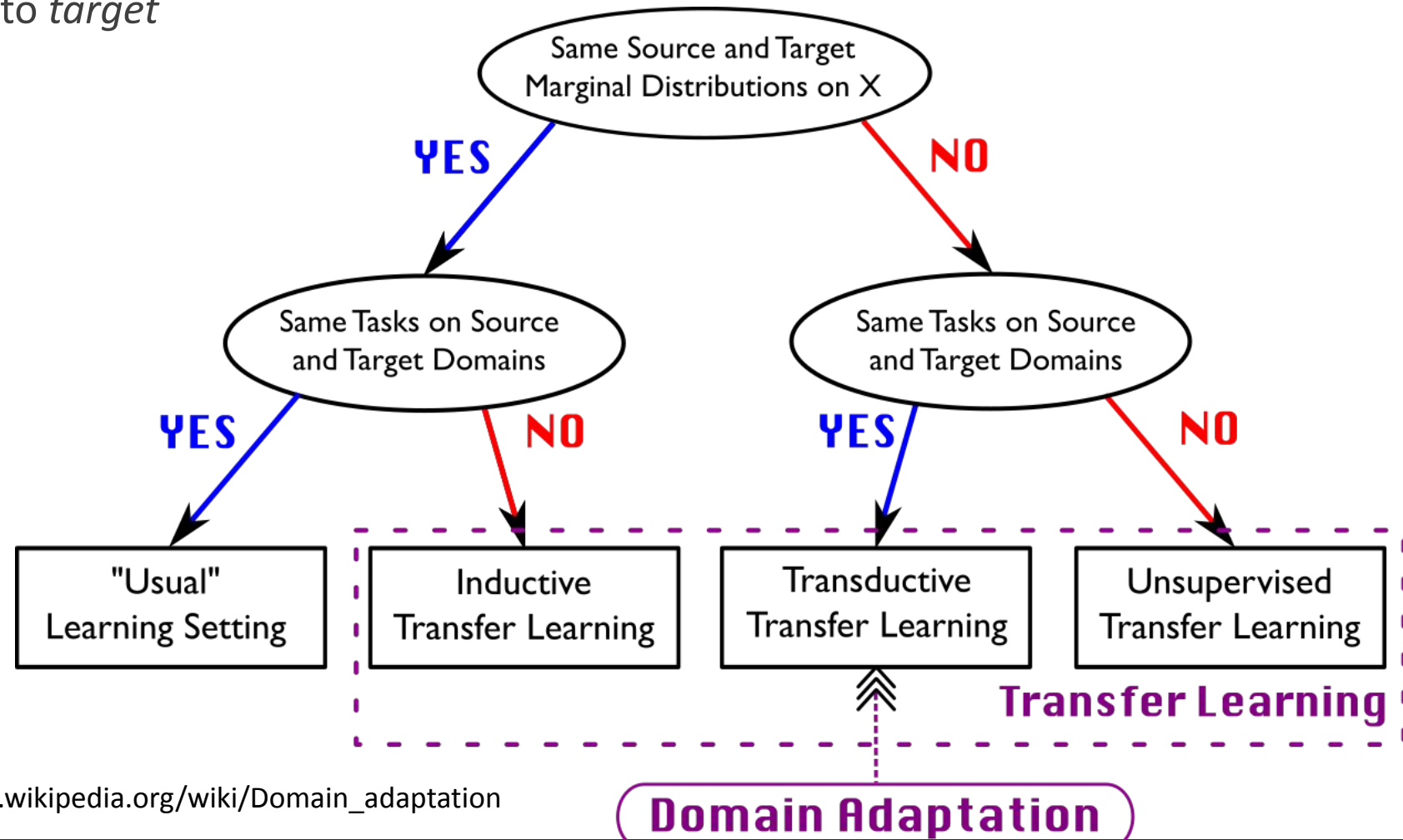
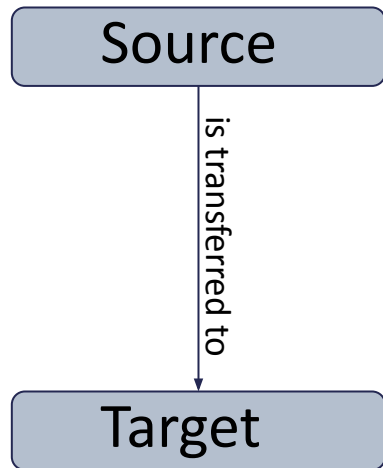
... but may not always be necessary



**Object detection**  
on MS COCO dataset  
123,287 images  
886,284 instances

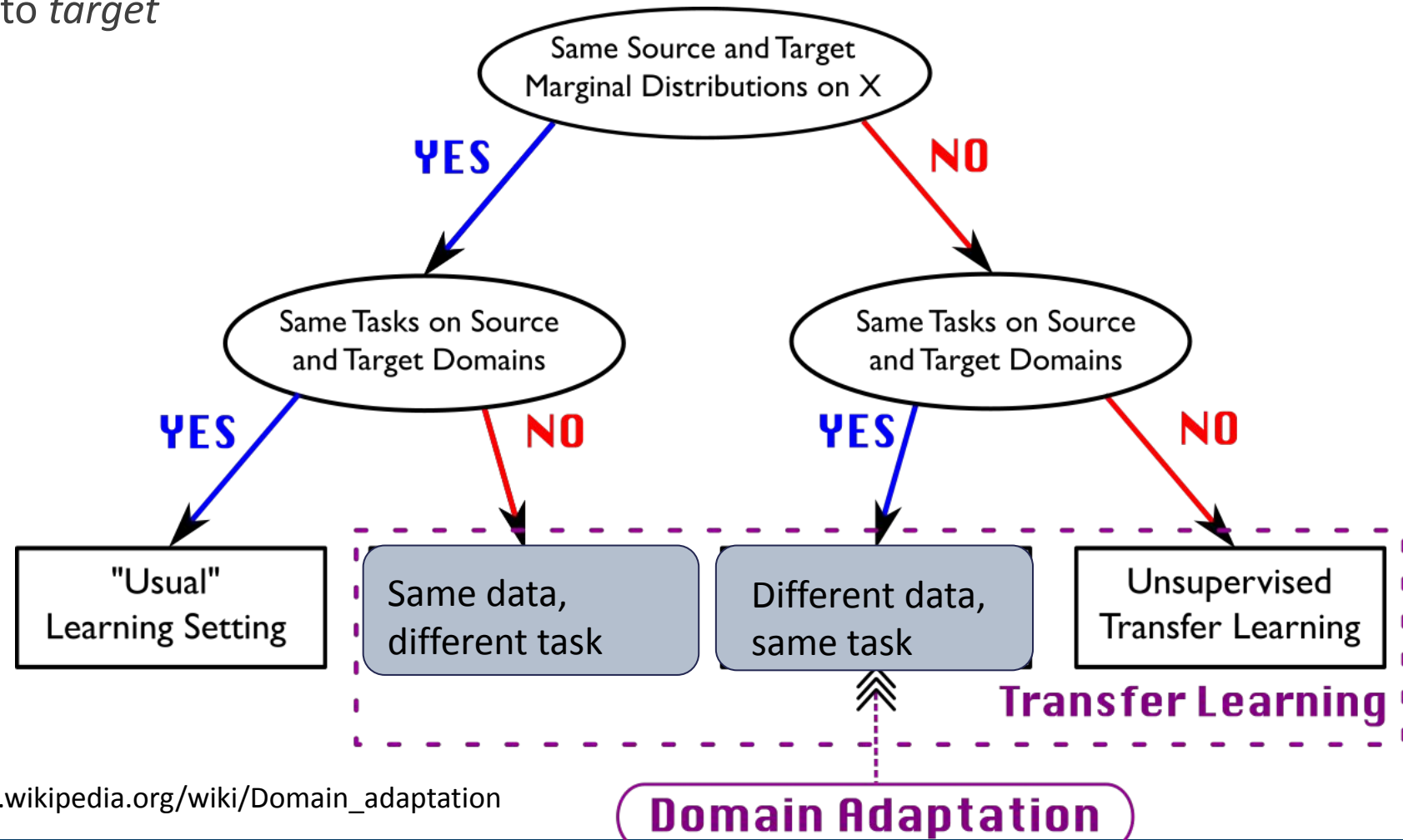
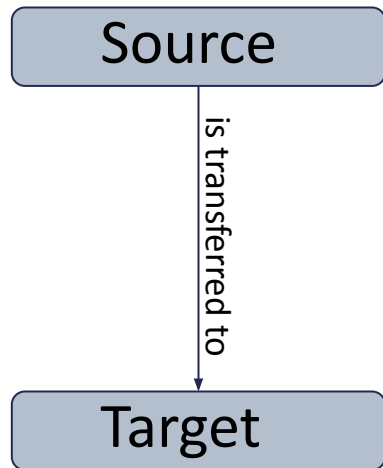
# Transfer learning categories

Transfer model from *source* to *target*



# Transfer learning categories

Transfer model from *source* to *target*



It's not just fine-tuning!

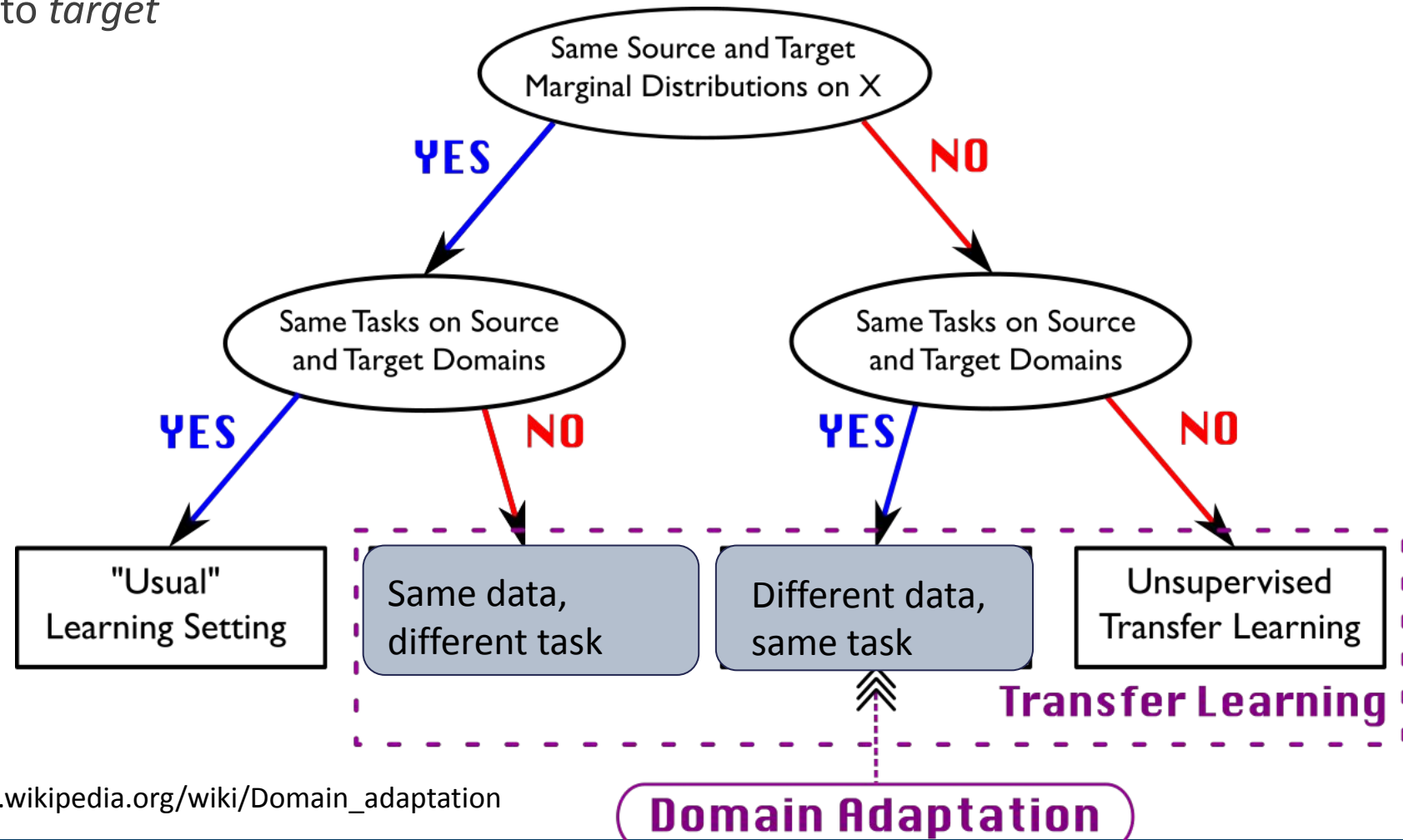
# Advanced transfer learning

Transfer model from *source* to *target*

Active area of research!

- Feature alignment
- Pseudo-labeling

We will revisit this in the context of semi-supervised learning.



# Visual saliency

---

PREDICTING WHERE PEOPLE LOOK

# Beyond Image Classification

---

So far: Image Classification (on ImageNet or Cifar)



# Beyond Image Classification

---

So far: Image Classification (on ImageNet or Cifar)

But: Understanding images requires much more than assigning a single label!





# Beyond Image Classification

So far: Image Classification (on ImageNet or Cifar)

But: Understanding images requires much more than assigning a single label!

Next lecture: Systematic overview of computer vision tasks.  
Most of them: use transfer learning (from ImageNet)

Rest of today: transfer learning from ImageNet for a specific task.



# Image saliency: predicting where people look

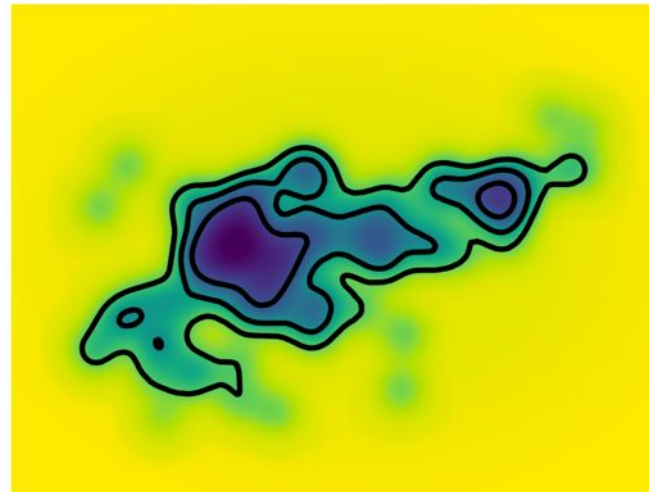
Image shown



Predict



Goal:  
Saliency map



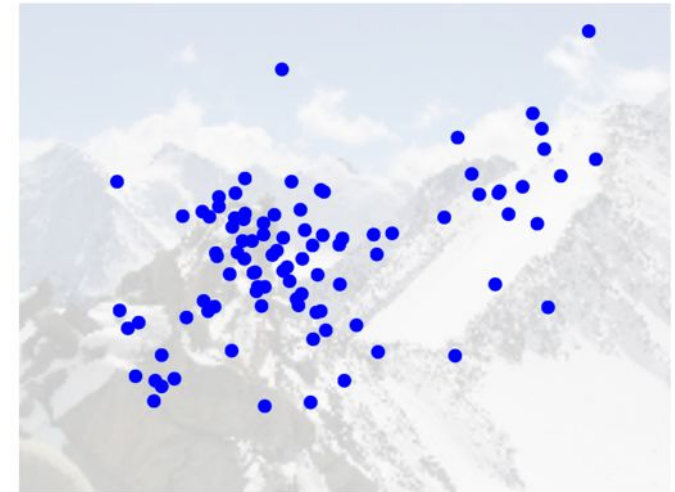
$$P(x, y \mid \text{image})$$

Low probability



High probability

Measured eye fixations



Loss



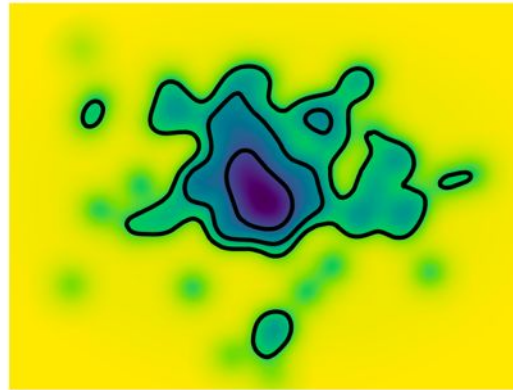
# Examples of saliency maps

---

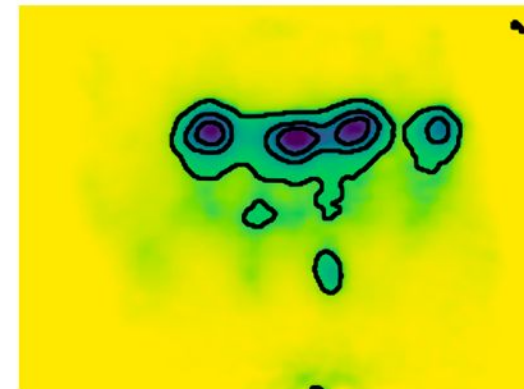
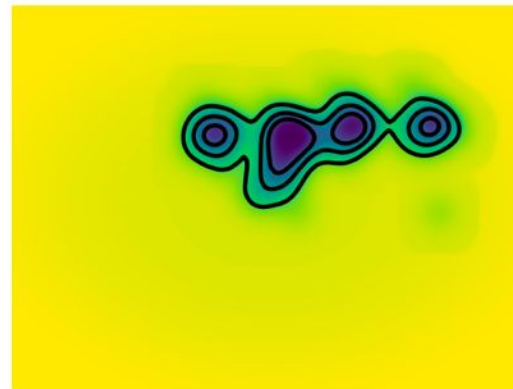
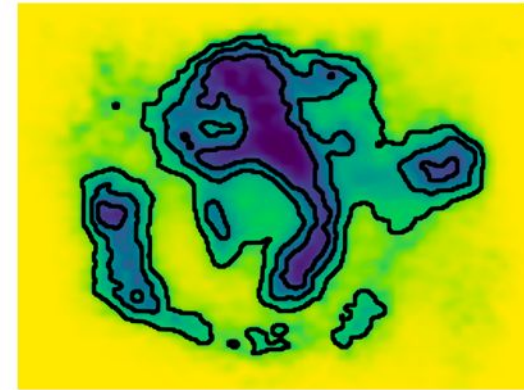
Image



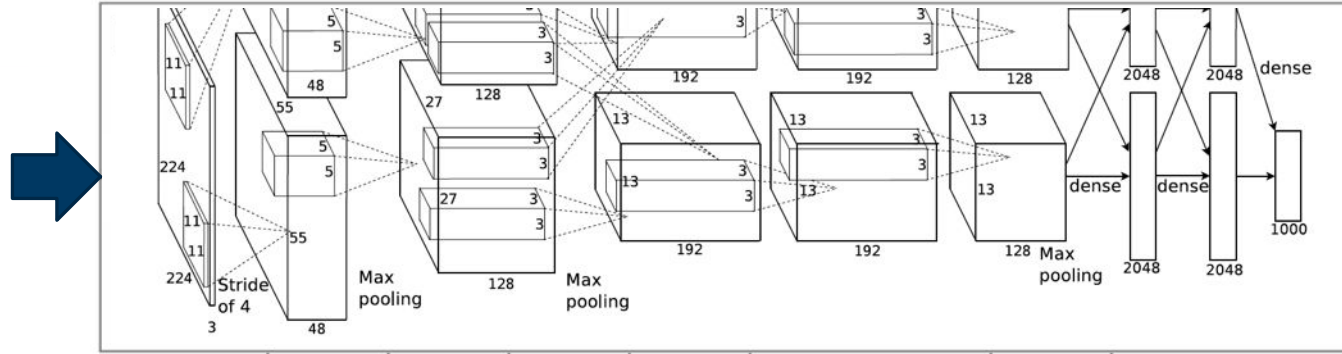
Ground truth (humans)



Model predictions







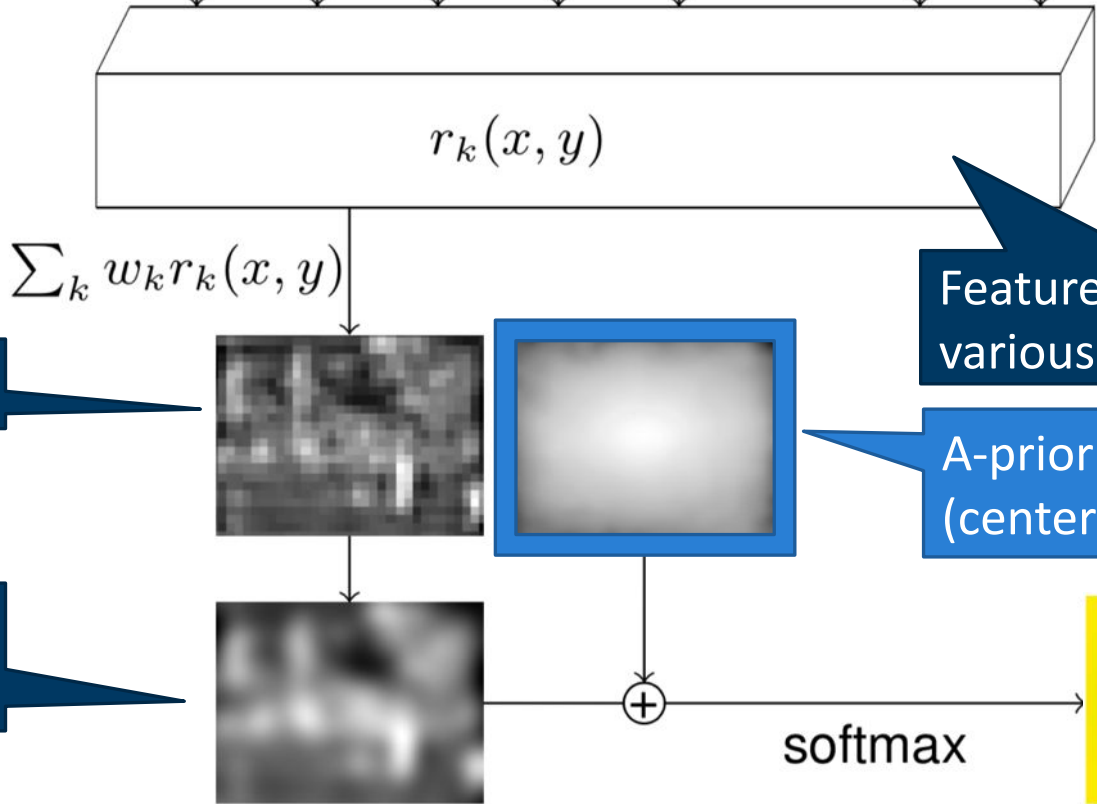
AlexNet  
pre-trained on  
ImageNet

# Deep Gaze I

Kümmerer, Theis, Bethge,  
ICLR Workshops 2015

Linear prediction

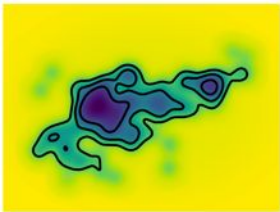
Blurring  
(as regularization)



Feature maps from  
various layers

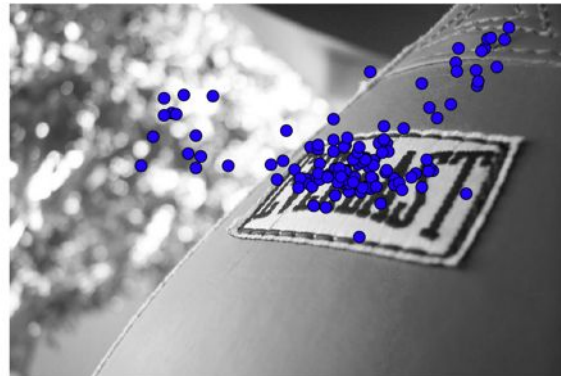
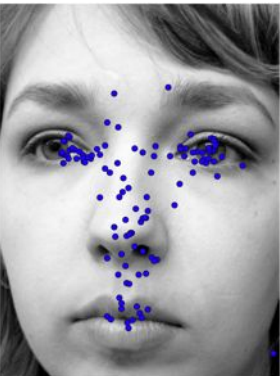
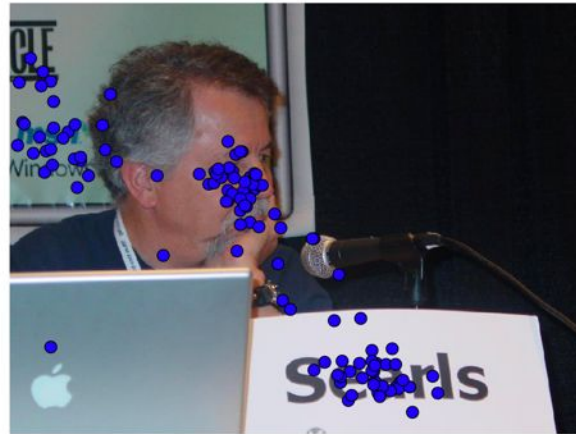
A-priori saliency map  
(center bias)

softmax



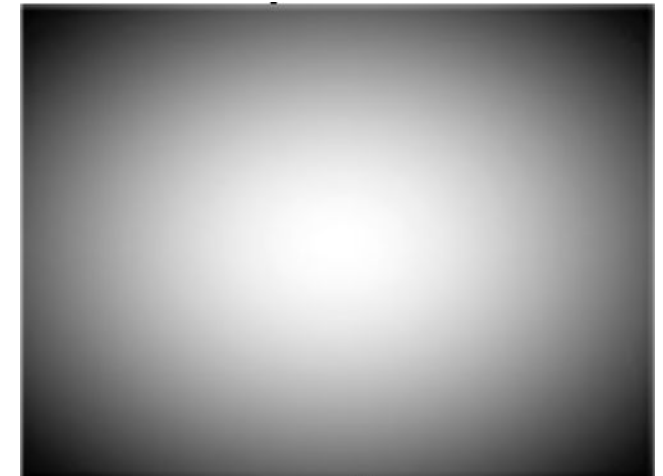
# Center bias

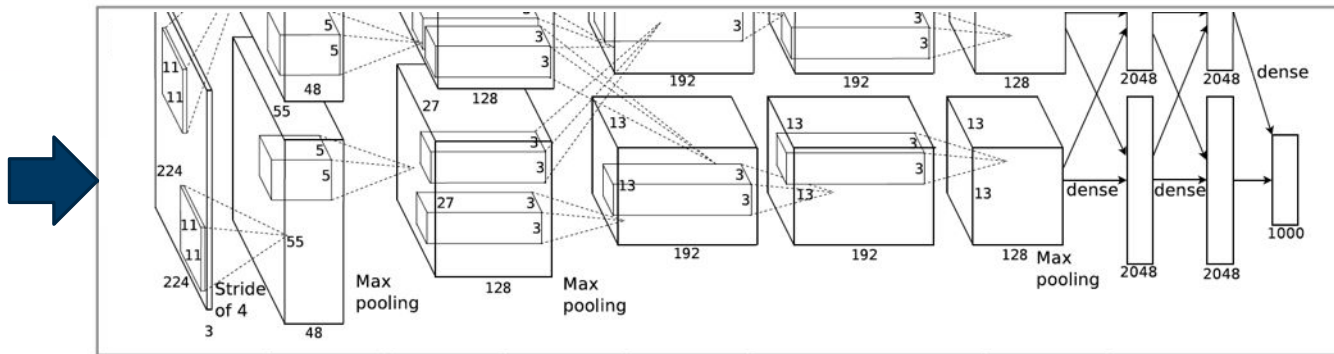
People tend to look at things in the center of the image



Convolutional net does not “care” where things are

-> Take into account prior

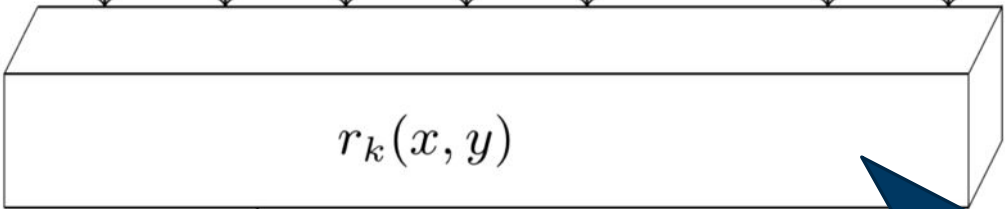




AlexNet  
pre-trained on  
ImageNet

# Deep Gaze I

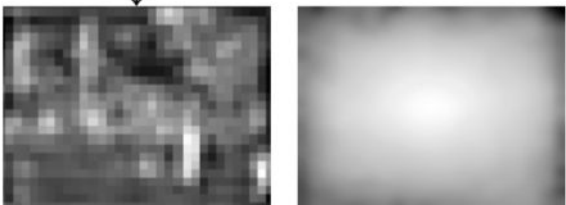
Kümmerer, Theis, Bethge,  
ICLR Workshops 2015



Feature maps from  
various layers

Linear prediction

$$\sum_k w_k r_k(x, y)$$

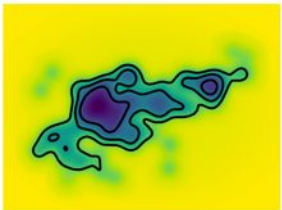


A-priori saliency map  
(center bias)

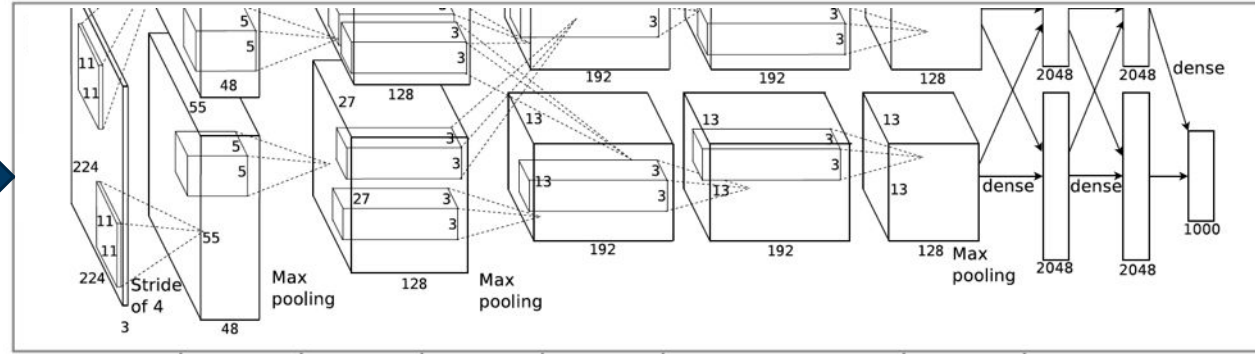
Blurring  
(as regularization)



softmax

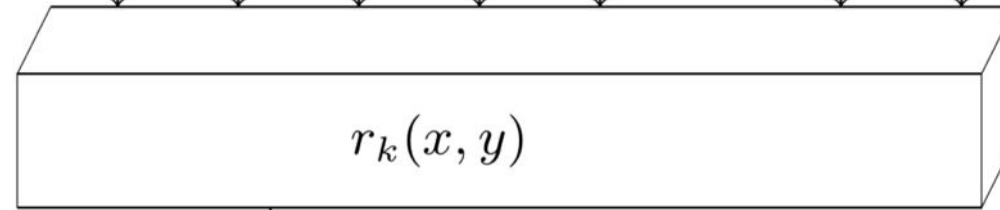


Saliency map  
(probability  
distribution)



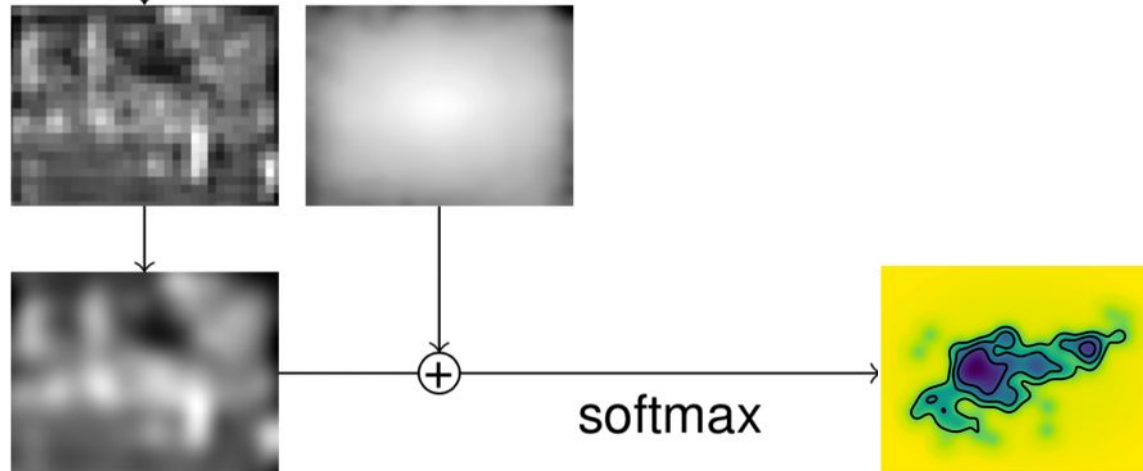
# Deep Gaze I

Kümmerer, Theis, Bethge,  
ICLR Workshops 2015



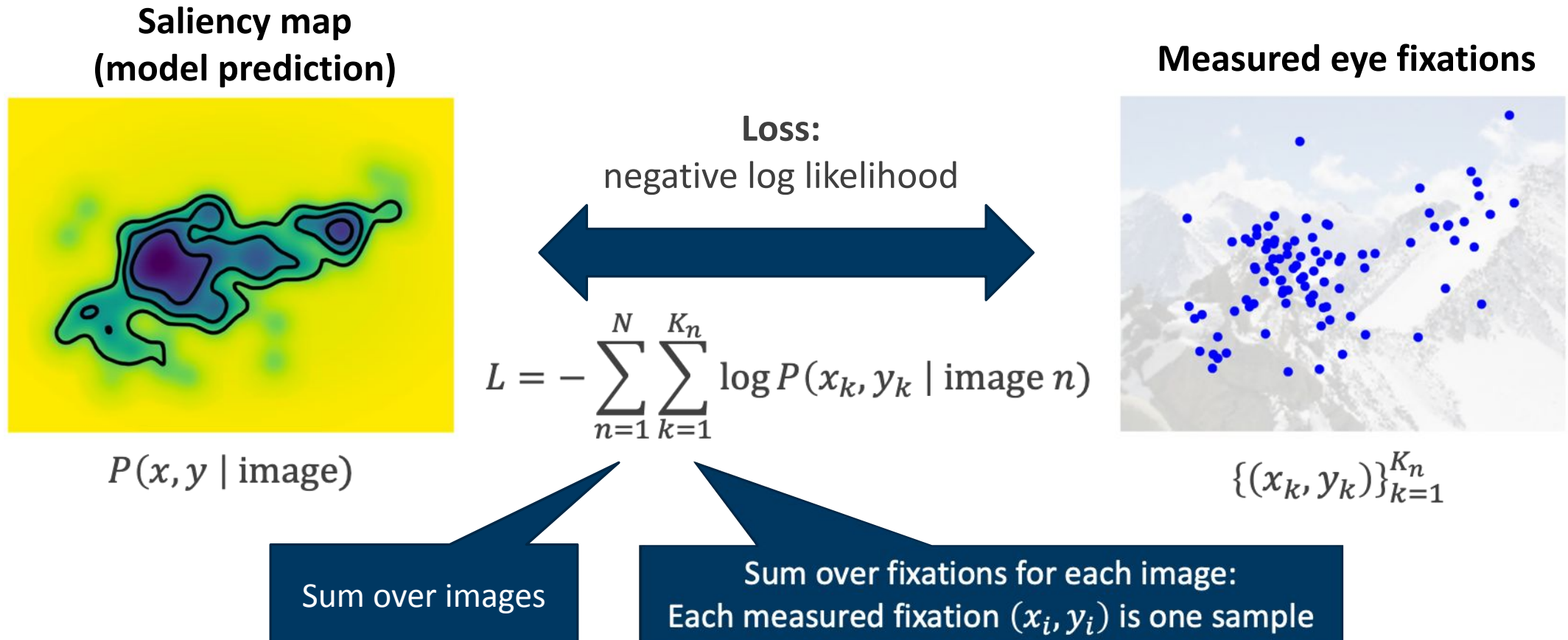
$$\sum_k w_k r_k(x, y)$$

Linear weights “w”  
are learned.  
**Transfer Learning!**





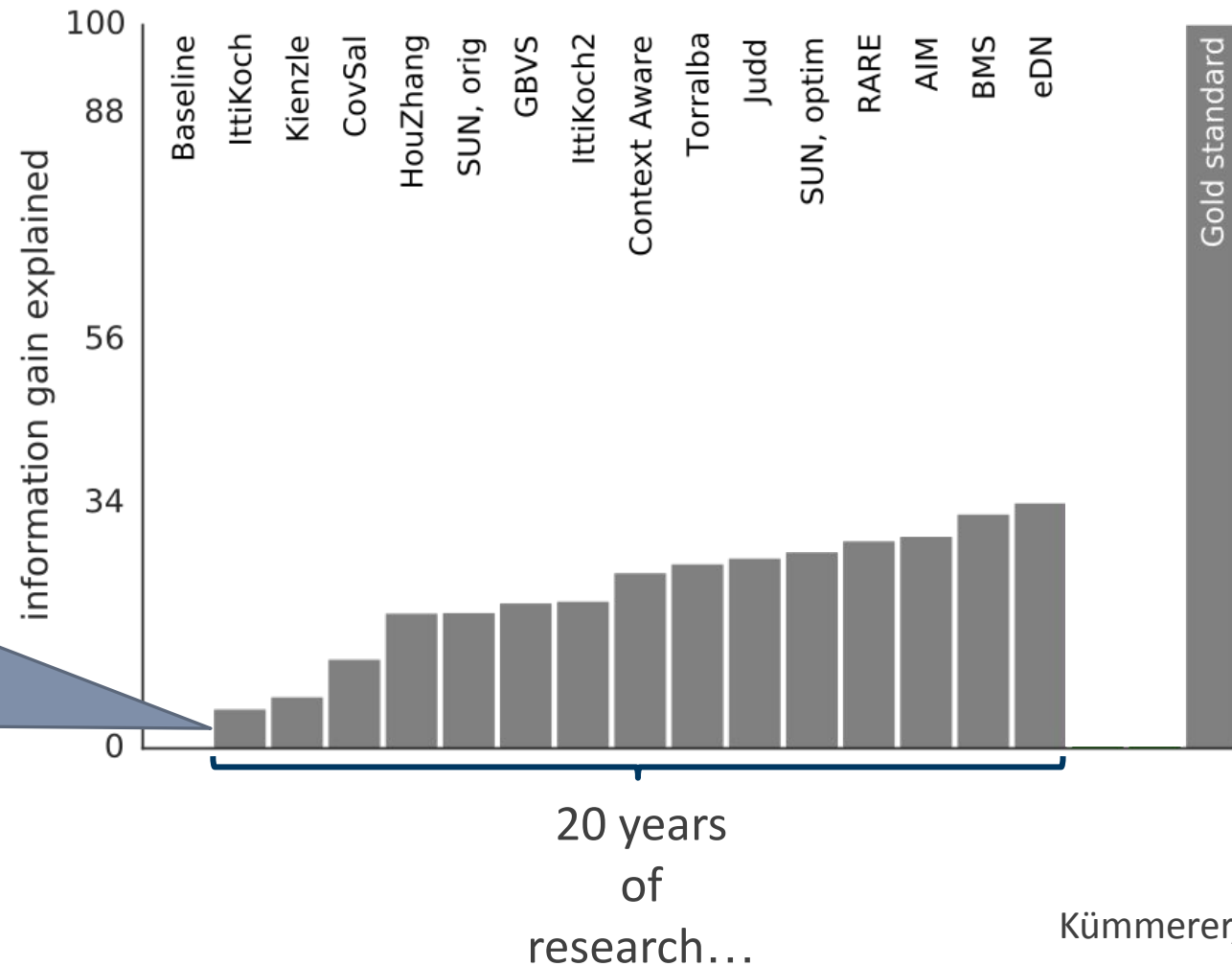
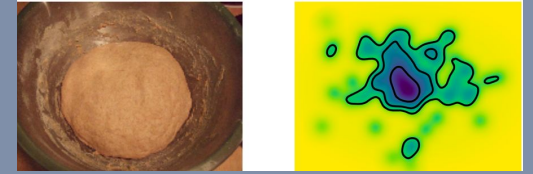
# Deep Gaze: loss function





# Deep Gaze I + II

Empirical distribution  
from other individuals

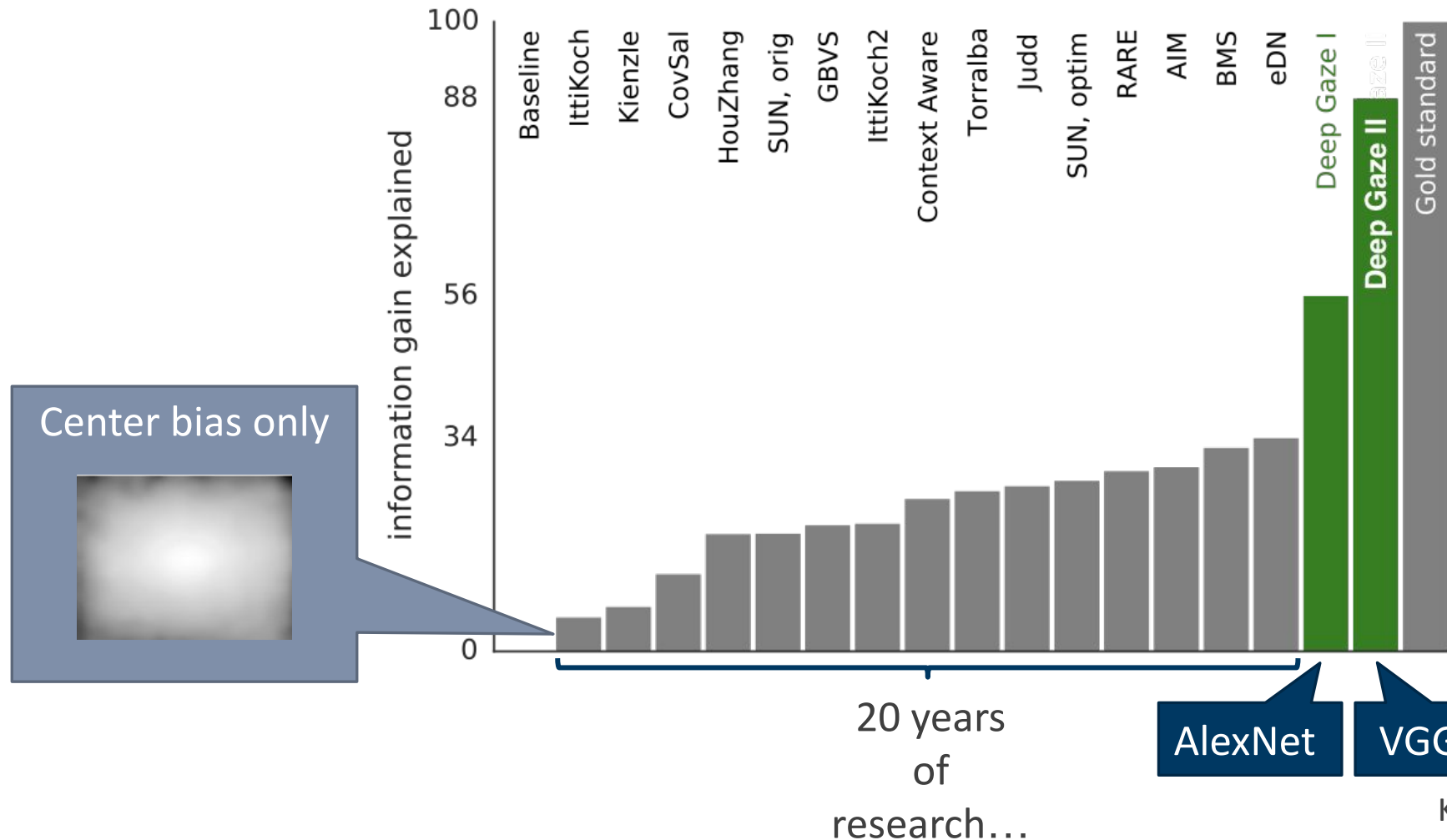
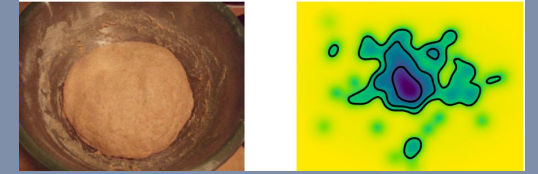


Center bias only



# Deep Gaze I + II

Empirical distribution  
from other individuals



Kümmerer, Theis, Bethge,  
*ICLR Workshops 2015*

Kümmerer, Wallis, Bethge, *arXiv 2016*

# Smart cropping by Twitter ☐ DeepGaze

Before: center crop



2 40

Now: neural network  
to predict saliency



2 40

# A cautionary tale

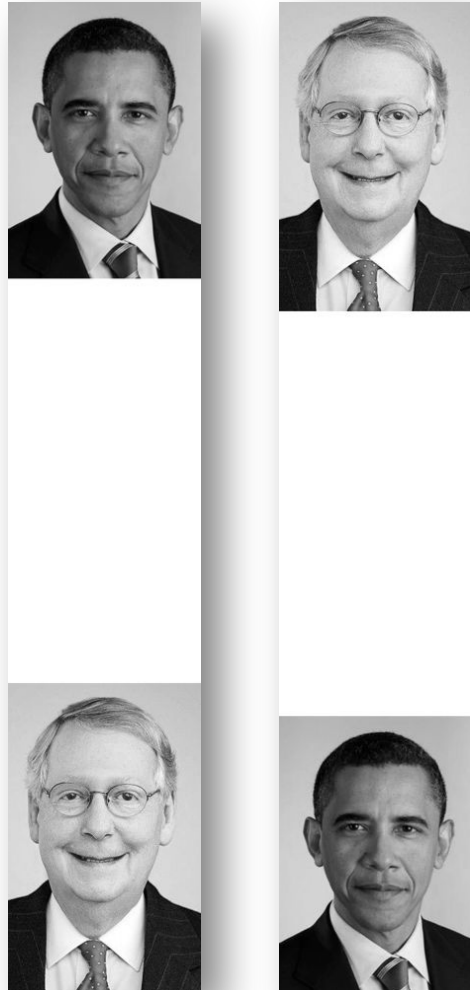
---

THE DOWNSIDE OF “SMART CROPPING”

# Beware: things don't always work as intended!

## What happens if we ask Twitter to crop these images?

One  
very  
long  
image



# What happened?

We don't know exactly, but probably any combination of the following:

- Black people do not appear as often in
  - ImageNet (the original dataset DeepGaze was trained on)
  - The gaze prediction dataset DeepGaze/Twitter autocrop was fine-tuned on
- The human subjects whose eye movements were measured actually looked more often at white people
- ...

**Beware of dataset bias!**



# Questions!

---